
How to Avoid Debate: Scalable AI Safety via Doubly-Efficient Interactive Proofs

Liyan Chen¹ Yael Tauman Kalai¹ Zoe Xi¹

Abstract

As AI models continue to develop powerful capabilities, it becomes critical that we are able to verify that their output is aligned with our intentions. A recent line of work focuses on verification via debate, a model of interactive proofs where two competing powerful provers, or AI models, debate each other to convince a weak verifier, or a human, of the correctness of their claim. However, debate assumes that the two AI models possess equal abilities and that one of them is truthful, which may not be realistic.

In this work, we show *how to avoid debate*: we initiate the study of *single-prover* interactive proofs for AI safety. Prior results in single-prover interactive proofs do not immediately carry over to the AI safety setting: for example, they do not work when the computation has access to an oracle, such as to human judgment or an external database such as the web. We present doubly-efficient single-prover interactive proofs and arguments for oracle-aided computations (also known as relativizing proofs), in the settings where (1) the computation is robust, in the sense that the output does not change if at most a small fraction of the answers to oracle queries are incorrect, or (2) the oracle is a low-degree polynomial. These results suggest that interactive verification is possible even without debate, under structured or noise-tolerant oracle access.

1. Introduction

As machine learning models become increasingly capable, they are being trained to perform complex tasks that would be prohibitively expensive for a human to verify, leading to

critical safety concerns. How can we efficiently supervise the training of an AI model to perform some complex task in a way that aligns with our intentions, when we may only have an obscure understanding of the task ourselves?

As a concrete example, consider the scenario of training a large language model (LLM) to write a long high-stakes document, such as a legal contract.¹ To generate a training label on a contract produced by the LLM, it is necessary to verify that every passage of the contract is correct, where correctness here is dictated by human judgment. However, it would be unreasonably expensive to require a human to carefully read through a long legal contract to produce just one training label. In this setting, it is important to have a training protocol that is extremely efficient in its use of human judgments. For another example, we might consider a scenario where we train an LLM on a large public dataset (e.g., the web). The LLM generates an output, and we would like to verify that the output is correct while making only very few queries to the dataset.

Various approaches to this problem, known as *scalable oversight*, have been proposed in the AI safety literature (Christiano et al., 2018; Leike et al., 2018; Irving et al., 2018). In particular, a recent line of work focuses on designing verifiable training protocols by leveraging tools from theoretical cryptography, namely the area of interactive proofs (Irving et al., 2018; Brown-Cohen et al., 2024; 2025). An interactive proof (Goldwasser et al., 1985; Babai, 1985) is a protocol wherein a powerful but untrusted prover interacts with a weak verifier to try to convince the verifier of the validity of some statement. For security, if the statement is indeed valid, then the verifier should accept, and if the statement is not valid, then the verifier should reject (with high probability). At a high level, taking the prover to be an AI model and the verifier to be a human, this closely resembles what we want in our safe training setting.

The main reason why prior work in interactive proofs does not immediately carry over to our setting, however, is that these classical results assume that the computation is given in a well-defined mathematical manner, such as a Turing

¹Massachusetts Institute of Technology. Correspondence to: Zoe Xi <zoxi@mit.edu>.

¹This example is due to Brown-Cohen, Irving, and Piliouras (Brown-Cohen et al., 2024).

machine. In the setting of AI safety, tasks can be modeled as computation with access to an oracle, which is a “black-box” function that we can’t see the inner workings of; an oracle might represent human judgment or an external database such as the web, for example. Going back to our examples, we would like to have a protocol where the verifier makes few queries to a human expert or to the dataset, which we could model as an oracle. Interactive proofs for oracle-aided computations are said to *relativize*, and we do not have interactive proofs that relativize. At the same time, relativization is believed to be essential to AI safety: the UK AISI Alignment team claims that “in order to be relevant to AI safety, an interactive proof result must relativize” (Irving & Marshall, 2025).

This aim of relativization has driven researchers to study a new model of interactive proofs, known as *debate*. In this model, introduced by Irving, Christiano, and Amodei in their 2018 paper “AI Safety via Debate” (Irving et al., 2018), there are two debating provers, who hold opposing claims, and a verifier, and the provers debate each other to try to convince the verifier of the validity of their claim. In the same paper, Irving, Christiano, and Amodei show that $\text{DEBATE} = \text{PSPACE}$ (where DEBATE is the class of problems that have a debate interactive proof), and this result relativizes, whereas the celebrated $\text{IP} = \text{PSPACE}$ result (Shamir, 1992) (where IP is the class of problems that have a single-prover interactive proof), does not; the moral is that to recover relativization one can introduce a second debating prover. A follow-up work by Brown-Cohen, Irving, and Piliouras (Brown-Cohen et al., 2024) introduces an analogue of this model of debate, called *doubly-efficient* debate, where the provers are constrained to run in polynomial time instead of being computationally unbounded—the motivation is that in our AI safety setting the provers are AI models, and AI models, while powerful, are still computationally bounded.

However, debate relies on assumptions that may not be realistic: in order to use debate for AI safety, we need that the two AI models are properly incentivized to debate, that they have the same computational abilities, and that one of them is truthful. This last assumption of truthfulness is particularly concerning: Irving, Christiano, and Amodei note that it hinges on the assumption that it is harder to lie than to refute a lie, that is, that one of the AI models is incentivized to be truthful, which may or may not be true in any particular setting (Irving et al., 2018). Further limitations of debate as an approach for AI alignment are discussed in (Irving et al., 2018).

In light of these limitations, in this work we investigate the following question:

Can we construct a single-prover interactive proof that relativizes?

In the “doubly-efficient” model that we study, there is a polynomial-time prover and an even more efficient verifier, who both have access to an oracle, and the prover tries to convince the verifier of the correctness of some computation that may depend on queries to the oracle. Notably, the super-efficient verifier not only runs in much less time than the prover, but he also makes few queries to the oracle, much fewer than the number that he would need in order to perform the computation by himself—in our examples, this would mean that very few queries to the human expert or to the database are needed.

Unfortunately, Barbara, Chiesa, and Guan (Barbara et al., 2025) prove that there do not exist interactive proofs that relativize for all computations. Intuitively, their observation is that if the computation uses many oracle gates to a random oracle, and the output of the computation is unstable, in the sense that it hinges on the answers of all the oracle queries being correct, then the verifier would need to check the correctness of every oracle call, and thus would not be efficient.

1.1. Our Results

In this paper, we construct relativizing doubly-efficient interactive proofs and arguments for two natural settings, where the computation is “robust” or where the oracle is a low-degree polynomial. Our results suggest we can recover relativization even without two debating provers.

Robust computation. We say that a computation is robust if the output does not change even if a small fraction of answers to oracle queries are modified. Intuitively, robustness allows a verifier to “spot-check” oracle calls instead of having to evaluate all of them. It seems natural to assume that the computation is robust, especially in the setting where the oracle is taken to represent human judgment: human judgment is already error-prone, so as a safeguard, the output of the computation should not change if a small number of the human answers change, otherwise the output is not very meaningful. More broadly, many natural tasks may be able to be made robust using redundancy, e.g., in the case of human judgment, we might rephrase the question or ask multiple people and take the majority answer.

Low-degree oracles. We also consider computations with access to an oracle that can be represented by a low-degree polynomial. Unlike the above robust setting, here we do not assume anything about the computation, but rely on the strong algebraic structure of the oracle in order to make the verifier efficient. Our protocol in this setting can be viewed as a first step towards designing protocols for computations with access to an oracle that is “learnable” by a simple machine learning model class. More concretely, we might take the oracle to represent a database, such as the web:

any database $\{0, 1\}^{\log N} \rightarrow \{0, 1\}$ can be converted into a low-degree oracle $\mathbb{F}^{\log N} \rightarrow \mathbb{F}$ via a standard low-degree extension (see Chapter 3 of (Thaler, 2022)). Then our protocol would allow us to efficiently verify the correctness of any computation that can interact with this database.

Our protocols exhibit different tradeoffs between soundness and efficiency guarantees. Some protocols achieve statistical soundness, which guarantees security against even computationally unbounded cheating provers, and some achieve only computational soundness, which guarantees security against polynomial-time cheating provers. The protocols have varying efficiency guarantees regarding the number of bits communicated between the prover and the verifier, the running time of the verifier, and the number of queries that the verifier makes to the oracle. For a more detailed explanation of these notions, see Section 3. Also see Table 1 for an overview of our results.

Outline of the paper. We use the main body of the paper to sketch our results and highlight some technical components, and defer the remainder of the paper to the appendix. We begin in Section 2 with an overview of the key ideas behind our results. In Section 3, we give part of our preliminaries, introducing our model of doubly-efficient interactive proofs for oracle-aided computation and some more essential definitions. We end the main body of the paper in Section 4 with some open questions. Within the appendix, in Section B we present the remainder of the preliminaries, and in Section C and Section D we give a formal presentation of our results for the settings where the computation is robust and the oracle is a low-degree polynomial, respectively.

1.2. Related Work

AI safety via debate. The work most closely related to ours is Brown-Cohen, Irving, and Piliouras’ “Scalable AI Safety Via Doubly Efficient Debate,” which presents doubly-efficient *debate* protocols for oracle-aided computation (Brown-Cohen et al., 2024); we study doubly-efficient *single-prover* protocols for oracle-aided computation. Debate was first proposed as an approach to scalable oversight in 2018 by Irving, Christiano, and Amodei in their paper “AI Safety via Debate” (Irving et al., 2018), which assumes that the two debating provers are computationally unbounded. More recently, work on debate has focused on limitations, e.g., the “obfuscated arguments problem,” wherein a dishonest debater can come up with a flawed argument where the flaw is very hard to find (Barnes, 2020; Brown-Cohen et al., 2025; Buhl et al., 2025).

Interactive proofs for trustworthy machine learning. Recently, there has been a line of work on applying interactive proofs to problems of trust in machine learning (Amit et al.,

2024; Hammond & Adam-Day, 2024; Anil et al., 2021; Wäldchen et al., 2024; Kirchner et al., 2024). The most closely related work in this line is that of Amit, Goldwasser, Paradise, and Rothblum (Amit et al., 2024) who introduce self-proving models, which are models that, in addition to providing answers, prove the correctness of those answers to a verifier using an interactive proof (with standard soundness and relaxed distributional completeness guarantees). Their focus is on training models to produce correctness proofs for computations that already admit interactive proofs. In contrast, our work expands the class of doubly efficient interactive proofs and arguments to oracle-aided computations.

Delegating computation. The problem of supervising complex computation with a weak verifier has been studied extensively in computational complexity and cryptography. Besides the celebrated $IP = PSPACE$ theorem (Shamir, 1992) and the PCP theorem (Arora & Safra, 1998), there has been a line of work studying doubly-efficient interactive proofs, motivated by the application of delegating computation (Goldwasser et al., 2015; Reingold et al., 2016; Berger et al., 2025). On the other hand, if we only consider security against polynomial-time algorithms, there has been a long line of work on constructing protocols for super-efficient verification for any NP statement, starting with the seminal work of Kilian and Micali (Kilian, 1992; Micali, 2000). These protocols, known as SNARGs or SNARKs, are already widely used in practice (see, e.g., (Ben-Sasson et al., 2014)). Our work can be regarded as an extension of both of the above lines, to the setting of oracle-aided computation.

2. Technical Overview

Here, let us sketch the key ideas behind our results.

In all of our protocols, we model the computation that the verifier wishes to verify as an oracle Boolean circuit, i.e., a Boolean circuit C that may contain gates that compute an arbitrary function $O : \{0, 1\}^* \rightarrow \{0, 1\}$ thought of as an oracle. We call such gates oracle gates and denote the circuit by C^O . Given input x , a succinct description of circuit C , and access to oracle O , the prover and verifier engage in a protocol to prove that $C^O(x) = 1$.

Throughout this overview, we use $\tilde{O}(\cdot)$ to hide polylogarithmic factors as well as, for simplicity, factors depending on the soundness parameter σ , the security parameter λ , and the oracle query length.

2.1. Proof system for robust oracle computation

A starting point: an IPP where the verifier has unreliable access to the input. Our proof system in the robust setting makes key use of an interactive proof of proximity (IPP) (Rothblum et al., 2013; Rothblum & Rothblum, 2020),

Table 1. An overview of our results. We model the computation as an oracle Boolean circuit of depth D with ℓ oracle gates divided into d levels. Here $\mathcal{V}_{\text{time}}$ is the verifier’s running time, $\mathcal{V}_q$ is the number of oracle queries made by the verifier, r is the round complexity, σ is a soundness parameter, and ε -robust means that the output of the circuit does not change even if at most an ε -fraction of the answers to oracle queries are changed on any input (see Section 3 for more formal definitions).

Protocol	Soundness	$\mathcal{V}_{\text{time}}$	$\mathcal{V}_q$	r	Setting	Adaptive?
Theorem C.1	statistical	$\tilde{O}(\sigma(\varepsilon\ell D + 1/\varepsilon + n))$	$O(\sigma/\varepsilon)$	$\tilde{O}(D)$	ε -robust	no
Theorem C.5	statistical	$\tilde{O}(\sigma d(\varepsilon\ell D + 1/\varepsilon + n))$	$\tilde{O}(\sigma d/\varepsilon)$	$\tilde{O}(dD)$	ε -robust	yes
Theorem C.10	computational	$\tilde{O}(1/\varepsilon + n)$	$O(1/\varepsilon)$	$O(1)$	ε -robust	yes
Theorem D.4	computational	$\tilde{O}(n)$	1	$\tilde{O}(1)$	poly-degree oracle	yes

which is a doubly-efficient interactive proof where the verifier is extremely efficient, running in only sublinear time in the input size, and the soundness guarantee is relaxed so that the verifier is only required to reject inputs that are “far” from any string in the language in, say, fractional Hamming distance. Notably, since the verifier runs in sublinear time, he cannot even read the entire input: in an IPP, the verifier is assumed to have query access to the input, meaning that he can read any bit in one timestep.

In our setting, we would like the verifier to make very few oracle queries, sublinear in the number of oracle gates in the circuit, which seems to resemble the IPP setting. One natural idea would be to have the prover and verifier engage in an IPP to show that $C_Q(x) = 1$, where Q consists of queries that C makes to O on input x along with the corresponding answers, and $C_Q(x)$ runs $C(x)$ but with the inputs and outputs of oracle gates replaced by those in Q . The verifier in this protocol would only have to read a few bits of Q , which is exactly what we want. Soundness follows from the relaxed soundness guarantee under the assumption that C is robust.

But this protocol doesn’t work as is, since we haven’t specified how the verifier accesses bits of Q . In an IPP, the verifier is assumed to have reliable access to Q (i.e., Q is written down in advance), but in our setting the verifier does not have reliable access to Q . Instead, at the end of their interaction,² we have the verifier ask the unreliable prover to send the bits in Q that he wishes to query, but since the prover might lie, we then have the verifier check that the prover’s answers are correct (or “close” to being correct). If the verifier requests position i in Q and the prover sends (x_i, y_i) , then the verifier can easily check that $y_i = O(x_i)$ by making one oracle call, but it might be inefficient for the verifier to check that x_i is computed correctly.

Recovering reliable access. In the case where the circuit C makes “nonadaptive” oracle queries, meaning that queries

²We assume that the verifier can make all of his queries to the input at the end of the interaction. This is indeed the case in both known IPP constructions in the literature (Rothblum et al., 2013; Rothblum & Rothblum, 2020).

are not allowed to depend on answers to other oracle queries, we can fix this problem by having the prover and verifier also engage in a doubly-efficient interactive proof (DEIP) to show that the x_i ’s that the prover sends are computed correctly from x . In the general setting where C may make adaptive oracle queries, i.e., queries may depend on answers to other oracle queries, this fix no longer works, since the computation that we would like to do a DEIP for may now contain oracle gates. Namely, in the nonadaptive setting, each x_i can only depend on x , while in the general setting, each x_i may also depend on the answers to several oracle queries, which may in turn depend on the answers to several oracle queries, and so on.

In the general setting, the idea is to instead combine this IPP idea with recursion. After the initial IPP to prove that $C_Q(x) = 1$, one natural idea is to do another IPP to prove that the prover’s message containing Q is computed correctly, and so on. As we explain below, this idea does not work as is, but does work if we add a minor tweak.

In what follows, we view the circuit C as containing d adaptivity levels of oracle gates, where queries can only depend on answers corresponding to oracle gates in smaller levels. Assume for simplicity that $d = 2$ (the argument for general d follows the same structure), and let Q_1 and Q_2 denote the substrings of Q corresponding to levels 1 and 2, respectively.

First attempt. After the initial IPP for proving that $C_Q(x) = 1$, the verifier queries the prover on some positions in Q , which we can separate into queries to Q_1 and queries to Q_2 , and the prover sends back answers. Letting A_2 denote the prover’s answers to Q_2 , the prover and verifier engage in an IPP to show that the queries in A_2 are computed correctly from x and Q_1 . At the end of this IPP, the verifier queries the prover for some positions in Q_1 , and the prover sends back answers. Letting A_1 denote the prover’s answers to Q_1 across both IPPs, the prover and verifier engage in a DEIP to show that the queries in A_1 are computed correctly from x . The verifier also checks himself that all the oracle calls in A_1 and A_2 are computed correctly.

This protocol doesn't work as is: the problem is that the computation that computes the answers for Q_2 is an oracle-aided computation that is not necessarily robust. Specifically, assuming that the input Q_1 to the second IPP is fixed (which in some sense is the case, since we check using a DEIP that A_1 is computed correctly from x), soundness of the second IPP only guarantees that there is some string Q'_1 close to Q_1 such that A_2 is computed correctly from Q'_1 and x , which means that there are potentially many different strings that the prover can choose to set A_2 to. So the input to the initial "IPP" is not fixed in advance (which means that the initial "IPP" is not in fact an IPP).

Our fix. To fix this, we have the prover commit (in a weak sense) to Q_1 and to Q_2 at the beginning of the protocol using a checksum: the prover computes checksums of Q_1 and Q_2 and sends these to the verifier. Then in each IPP, the prover additionally proves that the checksums of the inputs are correct: we change the initial IPP to prove that $C_Q(x) = 1$ and that the checksums of Q_1 and Q_2 are the same as the prover's checksums, and the second IPP to prove that A_2 is computed correctly from x and Q_1 and that the checksum of Q_1 is the same as the prover's checksum. In this $d = 2$ example, and for general d , we can show inductively that if the verifier accepts, then the input to each IPP is "fixed" assuming the inputs to IPPs corresponding to lower levels are "fixed," taking the IPP on input Q_1 as a base case, so that the "IPPs" in the protocol are in fact IPPs (albeit on inputs which may not be the claimed inputs, but are in some sense close to the claimed inputs). Under the assumption that the circuit is robust, the soundness guarantees of the IPPs and DEIP, the security of the checksum, and the verifier's checks together imply soundness.

Complexity analysis. The number of IPPs is equal to the adaptivity level, d , and each IPP corresponds to a depth $\leq D$ computation. Suppose that the circuit C is ε -robust, meaning that the output of C does not change even if at most an ε fraction of answers to oracle queries are changed (see Definition 3.3 for a formal definition). Using (Rothblum & Rothblum, 2020) (see Theorem B.5) and that C is ε -robust, we can implement each IPP to consist of at most $\tilde{O}(D)$ rounds, with query complexity $\tilde{O}(1/\varepsilon)$, communication complexity $\tilde{O}(\varepsilon \ell D)$, and verifier running time $\tilde{O}(\varepsilon \ell D + 1/\varepsilon + n)$. At the end, the prover and verifier run a DEIP corresponding to a depth $\leq D$ computation, which, using (Goldwasser et al., 2015) (see Theorem 3.5), consists of $\tilde{O}(D)$ rounds, with communication complexity $\tilde{O}(D)$ and verifier running time $\tilde{O}(D + n)$. Putting everything together, we get that the protocol consists of $\tilde{O}(dD)$ rounds, with query complexity $\tilde{O}(d/\varepsilon)$, communication complexity $\tilde{O}(\varepsilon \ell dD)$, and verifier running time $\tilde{O}(d(\varepsilon \ell D + 1/\varepsilon + n))$.

2.2. Argument system for robust oracle computation

We also give an argument system for robust oracle circuits that achieves better parameters than the proof system, by leveraging tools from cryptography. Recall that an argument system guarantees that soundness holds only against polynomial-time provers, while the soundness guarantee of a proof system must hold even against computationally unbounded provers (so essentially argument systems trade security for efficiency).

In our argument system, the prover uses a Merkle hash to commit to a string of queries and answers, which is supposed to be the string Q of queries and answers that C makes to O on input x . The prover and verifier engage in a succinct argument of knowledge (Kilian, 1992) to prove that there exists a string Q' such that (i) Q' is consistent with the commitment and (ii) $C_{Q'}(x) = 1$, where $C_{Q'}(x)$ runs $C(x)$ but with the inputs and outputs of oracle gates replaced by those in Q' . The additional knowledge property ensures that there not only exists a Q' satisfying (i) and (ii), but that the prover also knows such a Q' , which we can efficiently "extract" out. The verifier also asks the prover to open the commitment at a few random positions (specifically, at $O(1/\varepsilon)$ positions, where we assume the circuit C is " ε -robust") and checks that the oracle calls at these positions are correct. If the verifier accepts, then we can show that with high probability there is some fixed string Q' such that the computation "with respect to" Q' is correct, and most of the oracle calls in Q' are correct; then using the robustness assumption, we get soundness.

Using the "succinctness property" of the succinct argument of knowledge (Kilian, 1992) (see Theorem B.13), we get that the communication complexity of this protocol is dominated by the communication in the step where the prover opens the commitment at $O(1/\varepsilon)$ positions, which is $\tilde{O}(1/\varepsilon)$. Similarly, the verifier's running time is $\tilde{O}(1/\varepsilon + n)$. The query complexity is $O(1/\varepsilon)$, and the round complexity is $O(1)$.

2.3. Argument system for computation with access to a low-degree oracle

We also give an argument system for a different setting, where the circuit (which we no longer assume is robust) has access to an oracle that can be represented by a low-degree polynomial. In this protocol, we have the prover use a polynomial commitment scheme to commit to two polynomials G and F , which are supposed to encode the strings of queries and answers that C makes to O on input x , respectively. The prover uses a succinct argument of knowledge (Kilian, 1992) to convince the verifier that she knows G and F such that if we run $C(x)$ but with the queries and answers to O replaced with those encoded by G and F , then the output is 1. The verifier also asks the prover to

open the commitments at a random field point u and checks that $F(u) = O(G(u))$. Letting G^* and F^* denote the polynomials that we can extract out of the succinct argument of knowledge (using the knowledge property), if F^* does not correctly encode the oracle answers corresponding to G^* , then using that F^* and G^* are low-degree and applying Schwartz-Zippel, we get that w.h.p. $F^*(u) \neq O(G^*(u))$, so we achieve soundness. Notably, the verifier in this protocol only makes one query to the oracle.

Applying the guarantees of the succinct argument of knowledge (Kilian, 1992) (see Theorem B.13) and the polynomial commitment scheme (Cini et al., 2024) (see Theorem B.20) that we use, we get that this protocol has round complexity $\tilde{O}(1)$ and communication complexity $\tilde{O}(1)$. The verifier runs in time $\tilde{O}(n)$, which consists of the verification in Kilian’s protocol and the polynomial commitment scheme, and checking that $F(u) = O(G(u))$. The query complexity is 1.

3. Preliminaries

A *language* $\mathcal{L} \subseteq \{0, 1\}^*$ is a set of binary strings, and a family of circuits $\{C_n\}_{n \in \mathbb{N}}$, where for every n , C_n takes as input a binary string of length n and outputs a single bit, *decides* a language $\mathcal{L}$ if for every n and every $x \in \{0, 1\}^n$, we have $x \in \mathcal{L}$ if and only if $C_n(x) = 1$. A language is a formal description of a problem; if a circuit family decides a language, we can also think of it as solving the corresponding problem.

3.1. Oracle-Aided Computation and Robustness

In our protocols, we model the computation that the prover and verifier perform as an oracle Boolean circuit. Informally, a Boolean circuit is a diagram that shows how to obtain an output bit from a binary input string by applying some sequence of OR ($\vee$), AND ($\wedge$), and NOT ($\neg$) operations, and an oracle Boolean circuit with access to *oracle* $O : \{0, 1\}^* \rightarrow \{0, 1\}$ additionally can make calls to O .³ Here is a more formal definition:

Definition 3.1 ((Oracle) Boolean circuit). For every $n \in \mathbb{N}$, an n -input, single-output Boolean circuit is a directed acyclic graph with n *sources*, i.e., vertices with no incoming edges, and one *sink*, i.e., a vertex with no outgoing edges. Every non-source vertex is called a *gate* and is labelled with one of $\vee$, $\wedge$, or $\neg$; $\vee$ and $\wedge$ gates have two incoming edges and $\neg$ gates have one incoming edge. For some input string $x \in \{0, 1\}^n$, the output of the i th source vertex is the i th bit of x , and the value of a gate is defined recursively as the result of applying the logical operation of the gate on the

values of its *children*, i.e., the vertices with an edge going into the gate. The *output* of the circuit on x is the output of the sink vertex.

The *size* of a circuit is the number of gates it contains, and the *depth* is the length of the longest path from a source vertex to the sink vertex.

An oracle Boolean circuit with respect to oracle $O : \{0, 1\}^* \rightarrow \{0, 1\}$ additionally has oracle gates, which have an arbitrary number of incoming edges. We view the input to an oracle gate as a string given by the values of its children. The value of an oracle gate is the value of the oracle applied to its input string.

We say that an oracle circuit makes *adaptive* oracle queries if it contains two oracle gates that are connected by a path. This means that there is some oracle query that depends on the answer to another oracle query. An oracle circuit that does not make adaptive oracle queries makes *nonadaptive* oracle queries. A *d-adaptive* oracle circuit is one where the oracle gates are divided into d levels, and an oracle gate in level i can only be connected to oracle gates in levels $i + 1, \dots, d$, i.e., queries can depend only on answers corresponding to oracle gates in smaller levels. We can view any oracle circuit as d -adaptive: a circuit that makes nonadaptive oracle queries can be viewed as a circuit with one level of oracle gates, and a circuit that makes adaptive queries as a circuit with multiple levels.

Now we formally define what it means for an oracle circuit to be robust. Recall that what we want from this definition is that the output of a robust circuit should not change if at most a small fraction of the answers to its oracle queries are incorrect. Consider an n -input circuit C with access to oracle O ; we denote this by C^O . On input $x \in \{0, 1\}^n$, C^O makes a sequence of oracle queries and gets the corresponding answers, which we can denote by the string $Q = Q(x) = ((x_1, y_1), \dots, (x_\ell, y_\ell))$, where for each i , x_i is the i th oracle query and $y_i = O(x_i)$ the i th oracle answer. We call Q the *true query-answer* string. One could also imagine substituting Q with another length- ℓ string $Q' = ((x'_1, y'_1), \dots, (x'_\ell, y'_\ell))$, where for all i , x'_i is a binary string and y'_i is a bit, though it may not be the case that x'_i is computed correctly from x and previous oracle answers, or that $y'_i = O(x'_i)$ —we call a string of this form a *query-answer* string. For a query-answer string S , let us use S^{in} (resp. S^{out}) to denote the string of queries (resp. answers) in S . Then we could run the computation of C on x with “help” from Q'^{out} instead, i.e., replacing the oracle answers as given by Q^{out} with those given by Q'^{out} . We will denote this circuit by $C_{Q'}$. Note that $C^O(x) = C_Q(x)$.

We first define what it means for a true query-answer string and a query-answer string to be “close,” which we will then use to define robustness. Let Q be the true query-

³We assume for simplicity that O outputs a single bit. Our protocols can be readily extended to circuits with access to oracles that output multiple bits.

answer string for an oracle circuit C^O on input x , and write $Q = (Q_1, \dots, Q_d)$, where Q_i consists of all query-answer pairs in Q for oracle gates in the i th adaptivity level. We can decompose C^O level by level into circuits $C_0, \dots, C_d$ without oracle gates as follows: C_0 maps $x \mapsto Q_1^{\text{in}}$, for $i \in [d-1]$, C_i maps $(x, Q_1, \dots, Q_i) \mapsto Q_{i+1}^{\text{in}}$, and C_d maps $(x, Q_1, \dots, Q_d) \mapsto C^O(x)$.

Definition 3.2 (Closeness). Consider the following “ ε -closeness” algorithm: given a d -adaptive oracle circuit C^O with ℓ oracle gates, an input x , and the true query-answer string $Q = (Q_1, \dots, Q_d)$, we set $T_1 = Q_1$ and modify some positions in T_1 , obtaining a string Q'_1 . Then for $i = 2, \dots, d$, we set $T_i^{\text{in}} = C_{i-1}(x, Q'_1, \dots, Q'_{i-1})$ and $T_i = (T_i^{\text{in}}, O(T_i^{\text{in}}))$, where $O(T_i^{\text{in}})$ is obtained by applying O to each query in T_i^{in} , and modify some positions in T_i , obtaining a string Q'_i . If we can obtain $Q' = (Q'_1, \dots, Q'_d)$ via this algorithm by modifying at most $\varepsilon\ell$ positions (across all of the T_i ’s), then we say that Q' is ε -close to Q .⁴

Definition 3.3 (Robustness). We say that C^O is ε -robust if, for any length- n binary string x , $C_{Q'}(x) = C^O(x)$ for any Q' that is ε -close to the true query-answer string Q .

3.2. Interactive Proofs

An interactive proof is an interactive protocol between a weak verifier algorithm $\mathcal{V}$ and a powerful prover algorithm $\mathcal{P}$, where $\mathcal{P}$ tries to convince $\mathcal{V}$ of a statement of the form “ $x \in \mathcal{L}$.”

Definition 3.4 (Interactive proof system). An interactive proof for a language $\mathcal{L}$ is an interactive protocol between a probabilistic polynomial-time verifier algorithm $\mathcal{V}$ and a computationally unbounded prover algorithm $\mathcal{P}$. On common input x , $\mathcal{V}$ and $\mathcal{P}$ back-and-forth exchange messages in a number of rounds. In each round, $\mathcal{V}$ sends $\mathcal{P}$ a message and then $\mathcal{P}$ sends $\mathcal{V}$ a message. Both $\mathcal{V}$ ’s and $\mathcal{P}$ ’s messages can depend on x and any prior messages, and $\mathcal{V}$ ’s messages can additionally depend on $\mathcal{V}$ ’s random bits r . At the end of this interaction, their messages form a transcript $t = (\mathcal{V}(r), \mathcal{P})(x)$, and based on t , r , and x , $\mathcal{V}$ decides to accept or reject. The protocol satisfies completeness and soundness properties, namely,

- (Completeness.) For every $x \in \mathcal{L}$, there exists an honest prover strategy $\mathcal{P}$ such that $\Pr_r[V(x, t, r) = 1] = 1$, where $t = (\mathcal{V}(r), \mathcal{P})(x)$.
- (δ -Soundness.) For every $x \notin \mathcal{L}$ and for every (computationally unbounded) prover algorithm $\tilde{\mathcal{P}}$,

⁴Note that, when the circuit makes nonadaptive queries, this definition of closeness is exactly a fractional Hamming distance definition, i.e., Q and Q' are ε -close if we can turn Q into Q' by changing at most $\varepsilon\ell$ of its positions. We use this more involved definition because in circuits that make adaptive queries, changing the output of an oracle gate can affect what the “correct” input is to another oracle gate.

$$\Pr_r[V(x, \tilde{t}, r) = 1] \leq \delta, \text{ where } \tilde{t} = (\mathcal{V}(r), \tilde{\mathcal{P}})(x).$$

The *parameters* of interactive proofs that we are interested in include the prover’s running time, the verifier’s running time, the number of rounds (i.e., the round complexity) and the total number of bits communicated (i.e., the communication complexity).

In a standard interactive proof, the verifier is constrained to run in time polynomial in $|x| = n$ and the prover is computationally unbounded. A **doubly-efficient** interactive proof is one where the honest prover is constrained to run in polynomial time (though soundness still holds against computationally unbounded dishonest provers), and the verifier is even more efficient, running in near-linear time. In our setting of doubly-efficient interactive proofs for oracle computation, we additionally require that the verifier makes only a *sublinear* number of queries to the oracle.

We will make use of a construction of doubly-efficient interactive proofs due to Goldwasser, Kalai, and Rothblum:

Theorem 3.5 ((Goldwasser et al., 2015)). Let $\mathcal{L}$ be a language that has logspace-uniform Boolean circuits of depth $D = D(n)$ and size $S = S(n)$. For any soundness parameter $\sigma \in \mathbb{N}$, there is a doubly-efficient interactive proof for $\mathcal{L}$ with the following parameters:

- soundness error $2^{-\sigma}$;
- communication complexity $O(\sigma \cdot D \cdot \text{polylog}(S))$;
- round complexity $O(D \cdot \text{polylog}(S))$;
- verifier running time $O(\sigma \cdot (n + D \cdot \text{polylog}(S)))$; and
- prover running time $\text{poly}(S, \sigma)$.

Proof. The protocol of (Goldwasser et al., 2015) achieves the stated parameters with constant soundness error. Applying σ -fold parallel repetition reduces the soundness error to $2^{-\sigma}$, while preserving the round complexity and increasing the communication complexity and the prover’s and verifier’s running times by a factor of σ . $\square$

Interactive arguments. We also consider interactive *argument systems*, which are defined the same as interactive proof systems except that the soundness guarantee is only required to hold against provers that run in polynomial time. The soundness guarantee that a proof system is required to satisfy is called **statistical soundness**; argument systems are only required to satisfy **computational soundness**. Unlike statistical soundness, with computational soundness, a malicious prover $\tilde{\mathcal{P}}$ ’s success probability (i.e., the probability that $\mathcal{V}$ outputs 1) depends on the running time of $\tilde{\mathcal{P}}$. We give a formal definition in Section B.4.

4. Conclusion

In this paper, we initiated the study of single-prover interactive proofs for oracle-aided computations as an approach to scalable AI safety. While there do not exist interactive proofs for all oracle-aided computations, we presented relativizing doubly-efficient single-prover interactive proofs and arguments for two natural settings: (1) where the computation is robust and (2) where the oracle is a low-degree polynomial. We end on some interesting further directions of study.

- **Experimental validation.** One important direction for future work is experimental validation of our protocols. For example, it would be interesting to study which realistic scalable oversight tasks satisfy our robustness assumption and whether our protocols can be implemented efficiently in practice.
- **Other settings.** In this work, we considered the settings of robust computation and low-degree oracles. Are there other natural assumptions that one could make on the computation or on the oracle that would allow us to obtain relativizing single-prover interactive proofs? For example, it would be interesting to consider a broader class of “learnable” oracles.
- **Scope of robust computation.** As mentioned previously, many natural tasks may be able to be made robust using redundancy. Can we formalize this intuition as some theoretical model (and then apply our protocol for robust computation)?
- **Verifying tasks that we don’t know how to do.** In our model the verifier has access to the computation that he wants the prover to compute on some input; he is just too weak to perform the computation by himself. It would be interesting to study a model where not only can the verifier not perform the computation by himself, but he does not even have access to a description of the computation—this would, for example, capture the problem of scalable oversight in the setting where we are not able to perform the task that we are training the AI to do. Could we construct protocols for this setting?

Acknowledgments

The authors would like to acknowledge support provided by the UK AISI Alignment Project. Zoe Xi is supported by an Akamai Presidential Fellowship.

Impact Statement

This paper introduces formal approaches to enhancing factuality and safety in AI architectures, particularly Large

Language Models (LLMs). This represents a crucial area of machine learning research with significant practical implications. However, there is a significant gap between our theoretical understanding of these issues and real world practice, as current methods continue to rely heavily on expensive human feedback, which substantially limits their scalability.

References

- Amit, N., Goldwasser, S., Paradise, O., and Rothblum, G. Models that prove their own correctness. *arXiv preprint arXiv:2405.15722*, 2024.
- Anil, C., Zhang, G., Wu, Y., and Grosse, R. Learning to give checkable answers with prover-verifier games. *arXiv preprint arXiv:2108.12099*, 2021.
- Arora, S. and Safra, S. Probabilistic checking of proofs: A new characterization of np. *Journal of the ACM (JACM)*, 45(1):70–122, 1998.
- Babai, L. Trading group theory for randomness. In Sedgewick, R. (ed.), *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA*, pp. 421–429. ACM, 1985. doi: 10.1145/22145.22192. URL <https://doi.org/10.1145/22145.22192>.
- Barbara, A., Chiesa, A., and Guan, Z. Relativized succinct arguments in the ROM do not exist. In *TCC (I)*, volume 16268 of *Lecture Notes in Computer Science*, pp. 400–416. Springer, 2025.
- Barnes, B. Debate update: Obfuscated arguments problem, Dec 2020. URL <https://www.alignmentforum.org/posts/PJLABqQ962hZEqhdb/debate-update-obfuscated-arguments-problem>. AI Alignment Forum post.
- Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., and Virza, M. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pp. 459–474, Berkeley, CA, USA, May 18–21, 2014. IEEE Computer Society Press. doi: 10.1109/SP.2014.36.
- Berger, B., Goyal, R., Hong, M. M., and Kalai, Y. T. Efficiently batching unambiguous interactive proofs, 2025. URL <https://arxiv.org/abs/2510.19075>.
- Brown-Cohen, J., Irving, G., and Piliouras, G. Scalable ai safety via doubly-efficient debate. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 4585–4602, 2024.

- Brown-Cohen, J., Irving, G., and Piliouras, G. Avoiding obfuscation with prover-estimator debate. *arXiv preprint arXiv:2506.13609*, 2025.
- Buhl, M. D., Pfau, J., Hilton, B., and Irving, G. An alignment safety case sketch based on debate. *arXiv preprint arXiv:2505.03989*, 2025.
- Christiano, P., Shlegeris, B., and Amodei, D. Supervising strong learners by amplifying weak experts. *arXiv preprint arXiv:1810.08575*, 2018.
- Cini, V., Malavolta, G., Nguyen, N. K., and Wee, H. Polynomial commitments from lattices: Post-quantum security, fast verification and transparent setup. In Reyzin, L. and Stebila, D. (eds.), *Advances in Cryptology – CRYPTO 2024, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pp. 207–242, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. doi: 10.1007/978-3-031-68403-6_7.
- Goldwasser, S., Micali, S., and Rackoff, C. The knowledge complexity of interactive proof-systems (extended abstract). In Sedgewick, R. (ed.), *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, May 6-8, 1985, Providence, Rhode Island, USA*, pp. 291–304. ACM, 1985. doi: 10.1145/22145.22178. URL <https://doi.org/10.1145/22145.22178>.
- Goldwasser, S., Kalai, Y. T., and Rothblum, G. N. Delegating computation: interactive proofs for muggles. *Journal of the ACM (JACM)*, 62(4):1–64, 2015.
- Hammond, L. and Adam-Day, S. Neural interactive proofs. *arXiv preprint arXiv:2412.08897*, 2024.
- Irving, G. and Marshall, S. The need to relativise in debate, Jun 2025. URL <https://www.alignmentforum.org/posts/XycoFucvxAPhcgJQa/the-need-to-relativise-in-debate-1>. AI Alignment Forum post.
- Irving, G., Christiano, P., and Amodei, D. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- Kilian, J. A note on efficient zero-knowledge proofs and arguments. In *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, pp. 723–732, 1992.
- Kirchner, J. H., Chen, Y., Edwards, H., Leike, J., McAleese, N., and Burda, Y. Prover-verifier games improve legibility of llm outputs. *arXiv preprint arXiv:2407.13692*, 2024.
- Leike, J., Krueger, D., Everitt, T., Martic, M., Maini, V., and Legg, S. Scalable agent alignment via reward modeling: a research direction. *arXiv preprint arXiv:1811.07871*, 2018.
- Merkle, R. C. A certified digital signature. In Brassard, G. (ed.), *Advances in Cryptology – CRYPTO’89*, volume 435 of *Lecture Notes in Computer Science*, pp. 218–238, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA. doi: 10.1007/0-387-34805-0_21.
- Micali, S. Computationally sound proofs. *SIAM Journal on Computing*, 30(4):1253–1298, 2000.
- Reingold, O., Rothblum, G. N., and Rothblum, R. D. Constant-round interactive proofs for delegating computation. In Wichs, D. and Mansour, Y. (eds.), *48th Annual ACM Symposium on Theory of Computing*, pp. 49–62, Cambridge, MA, USA, June 18–21, 2016. ACM Press. doi: 10.1145/2897518.2897652.
- Rothblum, G. N. and Rothblum, R. D. Batch verification and proofs of proximity with polylog overhead. In *Theory of Cryptography Conference*, pp. 108–138. Springer, 2020.
- Rothblum, G. N., Vadhan, S., and Wigderson, A. Interactive proofs of proximity: delegating computation in sublinear time. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pp. 793–802, 2013.
- Shamir, A. $Ip = pspace$. *J. ACM*, 39(4):869–877, October 1992. ISSN 0004-5411. doi: 10.1145/146585.146609. URL <https://doi.org/10.1145/146585.146609>.
- Thaler, J. Proofs, arguments, and zero-knowledge. *Found. Trends Priv. Secur.*, 4(2-4):117–660, 2022. doi: 10.1561/33000000030. URL <https://doi.org/10.1561/33000000030>.
- Wäldchen, S., Sharma, K., Turan, B., Zimmer, M., and Pokutta, S. Interpretability guarantees with merlin-arthur classifiers, 2024. URL <https://arxiv.org/abs/2206.00759>.

A. Organization of Appendices

These appendices contain the technical parts of this work. In Section B we present the remainder of our preliminaries, and in Section C and Section D we present our results for the settings of robust computation and low-degree oracles, respectively.

B. Preliminaries

B.1. Succinct Descriptions of Sets and Functions

We define a notion of succinct representation of circuits. Loosely speaking, a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ has a succinct representation if there is a short string $\langle f \rangle$ of poly-logarithmic length that describes f . That is, $\langle f \rangle$ can be expanded to a full description of f . The actual technical definition is slightly more involved and in particular requires that the full description of f be a logarithmic-depth (i.e. NC^1) circuit:

Definition B.1 (Succinct Description of Functions). We say that a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ of size s has a succinct description if there exists a string $\langle f \rangle$ of length $\text{polylog}(n)$ and a logspace Turing machine M (of constant size, independent of n) such that on input 1^n , the machine M outputs a full description of an NC^1 circuit C such that for every $x \in \{0, 1\}^n$ it holds that $C(\langle f \rangle, x) = f(x)$. We refer to $\langle f \rangle$ as the succinct description of f .

We also define a notion of succinct representation for sets $S \subseteq [k]$. Roughly speaking this means that the set can be described by a string of length $\text{polylog}(k)$. The formal definition is somewhat more involved:

Definition B.2 (Succinct Description of Sets). We say that a set $S \subseteq [k]$ of size s has a succinct description if there exists a string $\langle S \rangle$ of length $\text{polylog}(k)$ and a logspace Turing machine M such that on input 1^k , the machine M outputs a full description of a depth $\text{polylog}(k)$ and size $\text{poly}(s, \log k)$ circuit (of constant fan-in) that on input $\langle S \rangle$ outputs all the elements of S as a list (of length $s \cdot \log(k)$).

We emphasize that the size of the circuit that M outputs is proportional to the actual size of the set S , rather than the universe size k .

B.2. Interactive Proofs of Proximity

Our proof systems for robust circuits make key use of an interactive proof of proximity (IPP). Loosely speaking, an IPP is a doubly-efficient interactive proof where the verifier is extremely efficient, running in only sublinear time, and the soundness requirement is weakened so that the verifier is only required to reject inputs that are “far” from the language with high probability, for, e.g., a fractional Hamming distance notion of distance. We will deal with IPPs for *pair languages*, where the input to the verifier is a pair (x, Q) consisting of an *explicit input* x that the verifier has direct access to and an *implicit input* Q that the verifier has query access to, i.e., the verifier treats Q as an oracle. We say that (x, Q) is ε -*Hamming-far* from pair language $\mathcal{L}$ for *proximity parameter* $\varepsilon \in (0, 1]$ if Q differs in at least an ε fraction of its positions from every Q' such that $(x, Q') \in \mathcal{L}$, and that it is ε -*Hamming-close* otherwise.

We denote the random variable of the outcome of an IPP as $\langle \mathcal{P}(x), \mathcal{V}^Q(x) \rangle$, where the randomness is over the randomness of $\mathcal{P}$ and $\mathcal{V}$.

Definition B.3 (Interactive proof of proximity (Rothblum et al., 2013; Rothblum & Rothblum, 2020)). An interactive proof of proximity for a pair language $\mathcal{L}$ is an interactive protocol between a probabilistic sublinear-time verifier algorithm $\mathcal{V}$ and a polynomial-time prover algorithm $\mathcal{P}$. Both $\mathcal{V}$ and $\mathcal{P}$ have access to explicit input x and a proximity parameter ε , and $\mathcal{V}$ has query access to implicit input Q while $\mathcal{P}$ has direct access to Q . The protocol satisfies completeness and a relaxed notion of soundness, namely

- (Completeness.) For every $(x, Q) \in \mathcal{L}$ and proximity parameter $\varepsilon \in (0, 1]$, there exists an honest prover strategy $\mathcal{P}$ such that $\Pr[\langle \mathcal{P}(x), \mathcal{V}^Q(x) \rangle = 1] = 1$.
- (δ -Soundness.) For every (x, Q) that is ε -Hamming-far from $\mathcal{L}$, and for every (computationally unbounded) prover algorithm $\tilde{\mathcal{P}}$, we have $\Pr_r[\langle \tilde{\mathcal{P}}(x), \mathcal{V}^Q(x) \rangle = 1] \leq \delta$.

Remark B.4 (Input Encoding). In IPP, the input can be treated as a binary string, and it can also be regarded as an element in some finite field $\mathbb{F}$. In the latter case, the communication and verifier’s running time also grow with $\log |\mathbb{F}|$, assuming addition and multiplication are efficient in that field.

The best-known IPP construction is given by Rothblum & Rothblum (2020).

Theorem B.5 ((Rothblum & Rothblum, 2020)). Let $\varepsilon = \varepsilon(m) \in (0, 1]$ be a proximity parameter, let $\sigma \in \mathbb{N}$ be a soundness parameter, and let $\mathcal{L}$ be a pair language that is computable by logspace-uniform Boolean circuits of depth $D = D(m) \geq \log(m)$ and size $S = S(m) \geq m$ with fan-in 2, where we use n for the length of the explicit input and m for the length of the implicit input. Then there is a public-coin IPP with ε proximity for $\mathcal{L}$ with the following parameters:

- soundness error $2^{-\sigma}$;
- query complexity $q = O(\sigma/\varepsilon)$;
- communication complexity $cc = O(\sigma \cdot \varepsilon \cdot m \cdot D \cdot \text{polylog}(S))$;
- round complexity $O(D \cdot \text{polylog}(S))$;
- verifier running time $O(\sigma \cdot (\varepsilon \cdot m \cdot D \cdot \text{polylog}(S) + (1/\varepsilon) \cdot \text{polylog}(S) + n \log S))$; and
- prover running time $\text{poly}(S, \sigma)$.

Furthermore, the verifier can make all of his queries to the implicit input at the end of the interaction. More formally, at the end of the interaction either the verifier rejects or in time $O(\sigma \cdot \varepsilon \cdot m \cdot D \cdot \text{polylog}(S))$ it outputs a succinct description $\langle Q \rangle$ of a set $Q \subseteq [m]$ of size q and a succinct description $\langle \phi \rangle$ of a predicate $\phi : \{0, 1\}^q \rightarrow \{0, 1\}$ so that its decision predicate given an implicit input x^{imp} is equal to $\phi(x_Q^{\text{imp}})$. Furthermore, the predicate can be evaluated in time $q \cdot \text{polylog}(m)$.

Proof. The theorem follows by composing Theorem 5.6 and Theorem 6.1 in (Rothblum & Rothblum, 2020) and applying σ -fold parallel repetition to obtain soundness error $2^{-\sigma}$. Parallel repetition preserves the round complexity, while increasing the query and communication complexities and the running times by a factor of $O(\sigma)$. $\square$

B.3. Unique-Decoding Checksums

Let $\rho \in \mathbb{N}$ be a *deviation radius* and $R_{\text{cksum}} \in \mathbb{N}$ be a checksum-length parameter. We use syndromes of linear error-correcting codes as checksums.

Definition B.6 (Unique-Decoding Checksums). Let $k, \rho, R_{\text{cksum}} \in \mathbb{N}$ and let $\mathbb{F}$ be a field. A function $\text{cksum}_\rho : \{0, 1\}^k \rightarrow \{0, 1\}^{R_{\text{cksum}}}$ is a ρ -unique decoding checksum function if for any $\mathbf{m} \in \{0, 1\}^k$, and for any $\mathbf{m}', \mathbf{m}'' \in \{0, 1\}^k$ that are both ρ -close to $\mathbf{m}$ and $\mathbf{m}' \neq \mathbf{m}''$, $\text{cksum}_\rho(\mathbf{m}') \neq \text{cksum}_\rho(\mathbf{m}'')$.

The term *unique decoding* refers to the fact that if we know $\mathbf{m}'$ is ρ -close to some (fixed) $\mathbf{m}$, then we can uniquely determine $\mathbf{m}'$ given $\text{cksum}_\rho(\mathbf{m}')$.

Proposition B.7 (Linear-code checksums). Let $\mathcal{C} \subseteq \mathbb{F}^k$ be a linear code of minimum distance D and codimension R_{cksum} , and $H \in \mathbb{F}^{R_{\text{cksum}} \times k}$ be a parity-check matrix for $\mathcal{C}$. Assume that $|\mathbb{F}| = 2^s$ and $\text{Enc} : \{0, 1\}^s \rightarrow \mathbb{F}$, $\text{Dec} : \mathbb{F} \rightarrow \{0, 1\}^s$ are linear maps converting between a binary string and the corresponding field element. For any $\rho \in \mathbb{N}$ satisfying $2\rho < D$, the syndrome map

$$\text{cksum}_\rho : \{0, 1\}^{ks} \rightarrow \{0, 1\}^{sR_{\text{cksum}}}, \quad \text{cksum}_\rho(\mathbf{m}) := \text{Dec}(H\text{Enc}(\mathbf{m})),$$

where Enc is applied blockwise and Dec coordinate-wise, is a ρ -unique decoding checksum function. Moreover, with $\tilde{O}$ hiding polylogarithmic factors, cksum_ρ can be evaluated in $\tilde{O}(kR_{\text{cksum}}s)$.

Proof. Fix $\mathbf{m} \in \{0, 1\}^{ks}$ and let $\mathbf{m}', \mathbf{m}'' \in \{0, 1\}^{ks}$ be distinct vectors that are both ρ -close to $\mathbf{m}$. Then $\mathbf{z} = \mathbf{m}' - \mathbf{m}''$ is nonzero and has Hamming weight at most 2ρ . If $\text{cksum}_\rho(\mathbf{m}') = \text{cksum}_\rho(\mathbf{m}'')$, then $\text{cksum}_\rho(\mathbf{z}) = \mathbf{0}$ by linearity, so $\text{Enc}(\mathbf{z}) \in \ker(H) = \mathcal{C}$. This is a nonzero codeword of Hamming weight at most $2\rho < D$, contradicting the minimum distance of $\mathcal{C}$. Hence $\text{cksum}_\rho(\mathbf{m}') \neq \text{cksum}_\rho(\mathbf{m}'')$. $\square$

Lemma B.8 (Reed–Solomon checksums). Let $k, \rho \in \mathbb{N}$ and let $\mathbb{F}$ be a finite field with $|\mathbb{F}| = 2^s \geq k$. Set $R_{\text{cksum}} := 2\rho$. Then a ρ -unique decoding checksum function $\text{cksum}_\rho : \{0, 1\}^k \rightarrow \{0, 1\}^{sR_{\text{cksum}}}$ exists. With $\tilde{O}$ hiding polylog(k) factors, $\text{cksum}_\rho : \{0, 1\}^k \rightarrow \{0, 1\}^{sR_{\text{cksum}}}$ can be evaluated in $\tilde{O}(kR_{\text{cksum}}s)$ time.

Proof. If $2\rho < k$, let $H \in \mathbb{F}^{2\rho \times k}$ be a parity-check matrix of a Reed–Solomon code over $\mathbb{F}$ with block length k , dimension $k - 2\rho$, and minimum distance $2\rho + 1$. Define $\text{cksum}_\rho(\mathbf{m}) := \text{Dec}(H\text{Enc}(\mathbf{m}||\mathbf{0}))$, where $\mathbf{0}$ is padding with appropriate length. By the previous proposition, this is a ρ -unique decoding checksum function.

If $2\rho \geq k$, let cksum_ρ be the identity map on $\{0, 1\}^k$ padded with $sR_{\text{cksum}} - k$ zero coordinates. Then cksum_ρ is injective, so it is ρ -unique decoding.

The evaluation bound follows by computing the R_{cksum} linear forms defining cksum_ρ . $\square$

B.4. Interactive Arguments and Arguments of Knowledge

We consider interactive arguments and arguments of knowledge.

Let $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^*$ be a ternary relation. If $(\text{pp}, x, w) \in \mathcal{R}$, we say that pp are the public parameters, x is a statement and w is a witness for x . Define the language $\mathcal{L}$ as $\{(\text{pp}, x) : \exists w \text{ s.t. } (\text{pp}, x, w) \in \mathcal{R}\}$.

Definition B.9 (Interactive argument system). Let $\ell \geq 0$ be an integer. A $(2\ell + 1)$ -message public-coin argument system $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for a relation $\mathcal{R}$ consists of a PPT (probabilistic polynomial time) algorithm Setup and a $(2\ell + 1)$ -message protocol between an interactive PPT prover $\mathcal{P}$ and an interactive PPT verifier $\mathcal{V}$ associated with a tuple $(X, W, (Z_{i-1}, C_i)_{i \in [\ell]}, Z_\ell)$, with the following properties:

- The Setup algorithm takes as input the security parameter 1^λ and outputs some public parameters pp .
- Both $\mathcal{P}$ and $\mathcal{V}$ receive as input the public parameters pp and a statement $x_0 = x \in X$. The prover $\mathcal{P}$ additionally receives a witness $w_0 = w \in W$.
- The public parameters pp , the statement x_0 , and the $2\ell + 1$ messages sent by $\mathcal{P}$ and $\mathcal{V}$ in the protocol, are collectively called a transcript, labelled as

$$(\text{pp}, x_0, z_0, c_1, \dots, z_{\ell-1}, c_\ell, z_\ell),$$

where $z_i \in Z_i$ is sent by $\mathcal{P}$ and $c_i \in C_i$ is sent by $\mathcal{V}$.

- The challenges c_i are sampled by $\mathcal{V}$ uniformly at random from C_i .

A transcript $(\text{pp}, x_0, z_0, c_1, \dots, z_{\ell-1}, c_\ell, z_\ell)$ is said to be accepting for Π if $\mathcal{V}(\text{pp}, x_0, z_0, c_1, \dots, z_{\ell-1}, c_\ell, z_\ell) = 1$ holds.

Now we define completeness, soundness and knowledge soundness.

Definition B.10 (Completeness). An argument system $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}$ has statistical completeness with correctness error ϵ if for all adversaries $\mathcal{A}$,

$$\Pr \left[b = 0 \wedge (\text{pp}, x, w) \in \mathcal{R} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\text{pp}) \\ (\text{tr}, b) \leftarrow \langle \mathcal{P}(\text{pp}, x, w), \mathcal{V}(\text{pp}, x) \rangle \end{array} \right] \leq \epsilon(\lambda).$$

Furthermore, we say that Π satisfies perfect completeness if $\epsilon = 0$.

Definition B.11 (Soundness). An argument system $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for the relation $\mathcal{R}$ has computational soundness if for all stateful PPT adversary $\mathcal{P}^*$, there exists a function δ negligible in λ , such that:

$$\Pr \left[b = 1 \wedge (\text{pp}, x) \notin \mathcal{L} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ (x, \text{st}) \leftarrow \mathcal{P}^*(\text{pp}) \\ (\text{tr}, b) \leftarrow \langle \mathcal{P}^*(\text{pp}, x, \text{st}), \mathcal{V}(\text{pp}, x) \rangle \end{array} \right] \leq \delta(\lambda).$$

Definition B.12 (Knowledge soundness). An argument system $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ is knowledge sound with knowledge error κ for the relation $\mathcal{R}^*$ if there exists an expected PPT extractor $\mathcal{E}$ such that for any stateful PPT adversary $\mathcal{P}^*$:

$$\Pr \left[b = 1 \wedge (\text{pp}, x, w) \notin \mathcal{R}^* \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ (x, \text{st}) \leftarrow \mathcal{P}^*(\text{pp}) \\ (\text{tr}, b) \leftarrow \langle \mathcal{P}^*(\text{pp}, x, \text{st}), \mathcal{V}(\text{pp}, x) \rangle \\ w \leftarrow \mathcal{E}_{\mathcal{P}^*}(\text{pp}, x) \end{array} \right] \leq \kappa(\lambda).$$

Here, the extractor $\mathcal{E}$ has a black-box oracle access to the (malicious) prover $\mathcal{P}^*$ and can rewind it to any point in the interaction.

The classical Kilian’s protocol (Kilian, 1992) is an argument of knowledge for any NP language, i.e., $\mathcal{R} = (\perp, x, w)$ where (x, w) is a valid instance-witness pair for some NP relation. Furthermore, Kilian’s protocol has the following succinctness property: the communication complexity is $\text{poly}(\lambda)$, and the running time of the verifier is $\text{poly}(\lambda) \cdot |x|$.

Theorem B.13. Assuming the existence of collision-resistant hash functions, there exists a succinct argument of knowledge for any NP relation $\mathcal{R}$ with knowledge error negligible in the security parameter λ .

B.5. Hash Trees

Our argument system for robust oracle circuits makes use of a hash tree, which is a verifiable hash function that supports local openings (i.e., it allows a prover to commit to a list of elements and later open any individual element) that can be instantiated from any collision-resistant hash function (Merkle, 1990).

Definition B.14. A hash tree consists of a tuple of algorithms $(\mathcal{G}_{\text{ht}}, \mathcal{H}_{\text{ht}}, \mathcal{P}_{\text{ht}}, \mathcal{V}_{\text{ht}})$ with the following syntax:

- $\mathcal{G}_{\text{ht}}(1^\lambda, S) \rightarrow \text{hk}$. This is a randomized algorithm that takes as input a security parameter λ , and a space bound S , and outputs a hash key hk . We implicitly assume that hk includes $1^\lambda, S$.
- $\mathcal{H}_{\text{ht}}(\text{hk}, D) \rightarrow \text{rt}$. This is a deterministic algorithm that on input a hash key hk and a database $D \in \{0, 1\}^S$ outputs a root rt .
- $\mathcal{P}_{\text{ht}}(\text{hk}, D, i) \rightarrow \rho$. This is a deterministic algorithm that on input a hash key hk , a database $D \in \{0, 1\}^S$, an index $i \in [S]$, outputs an opening proof ρ . When a set $I \subseteq [S]$ is given as input, we interpret this as providing an opening proof for every $i \in I$.
- $\mathcal{V}_{\text{ht}}(\text{hk}, \text{rt}, i, v, \rho) \rightarrow b$. This is a deterministic algorithm that on input a hash key hk , a hash tree root rt , an index $i \in [S]$, a bit $v \in \{0, 1\}$, and an opening proof ρ , outputs a bit $b \in \{0, 1\}$ indicating whether to accept or reject the opening proof. When a set $I \subseteq [S]$ and values $v \in \{0, 1\}^{|I|}$ are given as input, we interpret this as verifying, for every $j \in [I]$, the proof for the value $v[j]$ at the j th smallest index in I .

We require $(\mathcal{G}_{\text{ht}}, \mathcal{H}_{\text{ht}}, \mathcal{P}_{\text{ht}}, \mathcal{V}_{\text{ht}})$ to satisfy the following properties:

- **Opening completeness.** For any $\lambda, S \in \mathbb{N}$, $i \in [S]$, database $D \in \{0, 1\}^S$, it holds that

$$\Pr \left[\begin{array}{l} \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, S) \\ \text{rt} = \mathcal{H}_{\text{ht}}(\text{hk}, D) \\ \rho = \mathcal{P}_{\text{ht}}(\text{hk}, D, i) \end{array} : \mathcal{V}_{\text{ht}}(\text{hk}, \text{rt}, i, D[i], \rho) = 1 \right] = 1.$$

- **Efficiency.** In the opening completeness experiment above for $S \leq 2^\lambda$, $\mathcal{G}_{\text{ht}}$ and $\mathcal{V}_{\text{ht}}$ run in $\text{poly}(\lambda)$ time, and $\mathcal{H}_{\text{ht}}$ and $\mathcal{P}_{\text{ht}}$ run in $S \cdot \text{poly}(\lambda)$ time. Also we require $|\text{hk}| \leq \text{poly}(\lambda)$ and $|\rho| \leq \text{poly}(\lambda)$.
- **Binding.** For any non-uniform polynomial-time algorithm $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ and polynomial S , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} v \neq v' \\ \wedge 1 = \mathcal{V}_{\text{ht}}(\text{hk}, \text{rt}, i, v, \rho) \\ \wedge 1 = \mathcal{V}_{\text{ht}}(\text{hk}, \text{rt}, i, v', \rho') \end{array} : \begin{array}{l} \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, S(\lambda)) \\ (\text{rt}, i, v, v', \rho, \rho') \leftarrow A_\lambda(\text{hk}) \end{array} \right] \leq \mu(\lambda).$$

Remark B.15. For convenience, we may take the characters in the database D as some finite set Σ instead of just binary bits. The definition and properties of hash trees can be easily adapted to this setting, with a $\log |\Sigma|$ overhead on all efficiency parameters.

B.6. Polynomial Commitments

Our argument system for the setting where the oracle is low-degree uses a special commitment scheme called a polynomial commitment, which allows a prover to commit to a degree-bounded polynomial and later open evaluations of the polynomial at specific points, along with (interactive) proofs that the evaluations are correct.

Definition B.16. A (non-interactive) commitment scheme over $\mathcal{M}$ is a tuple of polynomial-time probabilistic algorithms $\text{CM} = (\text{Setup}, \text{Commit}, \text{Open})$ with the following syntax.

- $\text{Setup}(1^\lambda, d) \rightarrow \text{pp}$: Sample public parameters given a security parameter λ and message length d .
- $\text{Commit}(\text{pp}, f) \rightarrow (C, \text{st})$: Use the public parameters pp to compute a commitment C to a message $f \in \mathcal{M}$ and an auxiliary state st .
- $\text{Open}(\text{pp}, C, f, \text{st}) \rightarrow b$: Takes public parameters pp , a commitment C , a message $f \in \mathcal{M}$, and an auxiliary state st , and outputs a bit b indicating whether C is a valid commitment to f under pp .

We require commitment schemes to satisfy the following completeness and (relaxed) binding properties.

Definition B.17 (Completeness). A commitment scheme $\text{CM} = (\text{Setup}, \text{Commit}, \text{Open})$ satisfies completeness if for all $\lambda, d \in \mathbb{N}$, and for every $f \in \mathcal{M}$

$$\Pr \left[\text{Open}(\text{pp}, C, f, \text{st}) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, d) \\ (C, \text{st}) \leftarrow \text{Commit}(\text{pp}, f) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Definition B.18 (Binding). A commitment scheme $\text{CM} = (\text{Setup}, \text{Commit}, \text{Open})$ satisfies relaxed binding if for every PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} f \neq f' \text{ with } f, f' \in \mathcal{M} \\ \wedge \\ \text{Open}(\text{pp}, C, f, \text{st}) = \text{Open}(\text{pp}, C, f', \text{st}') = 1 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, d) \\ (C, (f, \text{st}), (f', \text{st}')) \leftarrow \mathcal{A}(\text{pp}) \end{array} \right] \leq \text{negl}(\lambda).$$

Now we define an extractable polynomial commitment.

Definition B.19 (Extractable polynomial commitment scheme). Let $\text{PC} = (\text{Setup}_{\text{CM}}, \text{Commit}, \text{Open}, \text{Setup}_{\text{IP}}, \mathcal{P}, \mathcal{V})$ be a tuple of algorithms. PC is an extractable polynomial commitment scheme for function class $\mathcal{F}$ if

- $(\text{Setup}_{\text{CM}}, \text{Commit}, \text{Open})$ is a commitment scheme over the function class

$$\mathcal{M} := \mathcal{F}.$$

- $(\text{Setup}_{\text{IP}}, \mathcal{P}, \mathcal{V})$ is an argument system for the relation

$$(\text{pp}, (\text{pp}_{\text{CM}}, C, \mathbf{x}, \mathbf{u}), (f, \text{st})) \in \mathcal{R} \iff \text{Open}(\text{pp}_{\text{CM}}, C, f, \text{st}) = 1 \wedge f(\mathbf{x}) = \mathbf{u}.$$

The class of functions $\mathcal{F}$ supported by a polynomial commitment scheme will be a set of polynomials. We say that the polynomial commitment scheme satisfies completeness and knowledge soundness if $(\text{Setup}_{\text{IP}}, \mathcal{P}, \mathcal{V})$ is complete and knowledge sound respectively.

Theorem B.20 ((Cini et al., 2024)). Assuming the standard (Module)-SIS assumption, there exists an extractable polynomial commitment scheme with $\text{poly}(\lambda, \log L)$ communication and verification times, where L is the degree bound of the committed polynomial. Furthermore, the number of rounds of the local opening argument system is $O(\log L)$.

C. Doubly-Efficient Interactive Proofs for Robust Oracle Circuits

Here we present our doubly-efficient single-prover interactive proof and argument systems for robust oracle circuits. We first present a proof system for robust circuits that make nonadaptive oracle queries in Section C.1 below. In Section C.2, we extend this to a proof system for general robust circuits (that may make adaptive queries). In Section C.3, we give an argument system for general robust circuits.

C.1. A Proof System for Circuits Making Nonadaptive Oracle Queries

Theorem C.1. Let $\varepsilon = \varepsilon(n) \in (0, 1]$, $\sigma \in \mathbb{N}$, and let $\mathcal{L}$ be a language decidable by ε -robust logspace-uniform oracle Boolean circuits with $\ell = \ell(n)$ nonadaptive oracle gates, depth $D = D(n) \geq \log(n)$, and size $S = S(n) \geq n$, where all gates except possibly the oracle gates have fan-in 2. We assume that the query length is bounded by $m = \text{poly}(n)$; and $\varepsilon \geq 1/\ell$, as otherwise ε -robustness holds vacuously (see Definition 3.3). Then there is a doubly-efficient interactive proof system for $\mathcal{L}$ with the following parameters:

- soundness error $2^{-\sigma}$;
- query complexity $q = O(\sigma/\varepsilon)$;
- communication complexity $cc = O(\sigma \cdot m \cdot \varepsilon \cdot \ell \cdot D \cdot \text{polylog}(S) + (\sigma/\varepsilon) \cdot m \cdot \text{polylog}(\ell))$;
- round complexity $D \cdot \text{polylog}(S)$;
- verifier running time $O(\varepsilon \ell \cdot D + 1/\varepsilon) \cdot \sigma \cdot m \cdot \text{polylog}(S) + O(\sigma n \log S)$; and
- prover running time $\text{poly}(S, \sigma)$.

DEIP for robust, nonadaptive setting

Input: The prover $\mathcal{P}$ and verifier $\mathcal{V}$ receive $x \in \{0, 1\}^n$, a succinct description of an ε -robust logspace-uniform oracle circuit C with ℓ nonadaptive oracle gates, depth D , and size S , and oracle access to $O : \{0, 1\}^* \rightarrow \{0, 1\}$. Note that ε is a fixed parameter, while ℓ , D , and S are parameters of the input.

Additional parameters: We let σ denote the soundness parameter.

Notations: Enumerate the oracle gates by $1, \dots, \ell$. Let $Q = Q(x) = ((x_1, y_1), \dots, (x_\ell, y_\ell))$ denote the true query-answer string and Q^{in} (resp. Q^{out}) denote the strings of queries (resp. answers) in Q . We split C^O into two circuits without oracle gates: C_0 computes Q^{in} from x , and C_1 computes $C^O(x)$ from (x, Q) .

Ingredients: We make use of the following tools:

- An IPP for the pair language $\mathcal{L}_{\text{acc}} = \{(x, Q) : C_1(x, Q) = 1\}$ with proximity parameter ε and soundness parameter $\sigma + 2$ (instantiated by Theorem B.5).
- A DEIP for the language $\mathcal{L}_{J, A^{\text{in}}}^*$ defined below (which asserts correctness of $C_0(x)$ on a subset of coordinates), with soundness parameter $\sigma + 2$ (instantiated by Theorem 3.5).

The protocol:

1. The prover evaluates $C^O(x)$ and computes the true query-answer string Q . The prover and the verifier run the IPP for $\mathcal{L}_{\text{acc}}$ with proximity parameter ε on input (x, Q) , where x is the explicit input and Q is the implicit input. At the end of this interaction, the IPP verifier outputs the description $\langle J \rangle$ of indices $J \subseteq [\ell]$ that it wishes to query in Q and the description $\langle \phi \rangle$ of the predicate ϕ . The verifier sends $\langle J \rangle$ to the prover.
2. The prover sends $A := Q|_J$. The verifier finishes the IPP verification by checking $\phi(A) = 1$. The verifier also checks that the oracle calls in A are correct: for all $i \in J$, it checks that $y_i = O(x_i)$.
3. Letting $A^{\text{in}} = (x_i)_{i \in J}$ denote the string of queries in A , the prover and the verifier engage in a DEIP to prove that $C_0(x)|_J = A^{\text{in}}$. Formally, letting

$$\mathcal{L}_{J, A^{\text{in}}}^* = \{x \in \{0, 1\}^n : C_0(x)|_J = A^{\text{in}}\},$$

the prover and verifier run a DEIP to prove that x is in $\mathcal{L}_{J, A^{\text{in}}}^*$.

4. The verifier accepts if the IPP and DEIP verifiers accept and all of its checks pass, and rejects otherwise.

Figure 1. DEIP for robust circuits making nonadaptive queries

Proof. The protocol is given in Figure 1. We analyze its completeness, soundness, and efficiency.

Completeness. If $C^O(x) = 1$, then $(x, Q) \in \mathcal{L}_{\text{acc}}$. The honest prover answers the verifier's query in step 2 with $A = Q|_J$. Then the verifier's oracle correctness checks pass, the DEIP claim $C_0(x)|_J = A^{\text{in}}$ holds, and by completeness of the IPP and DEIP the verifier accepts with probability 1.

Soundness. Assume that $x \notin \mathcal{L}$ and fix any dishonest prover strategy $\tilde{\mathcal{P}}$. Let J be the set of indices queried by the IPP verifier in step 1, ϕ be the predicate the IPP verifier outputs, Q be the true query-answer string (that is, the query-answer string obtained from an *honest* evaluation of $C^O(x)$), and $\tilde{A} = \{(\tilde{x}_i, \tilde{y}_i) : i \in J\}$ be the prover's response in step 2. Here Q is a fixed string determined by x , while J , ϕ , and $\tilde{A}$ are random variables determined by the interaction $\langle \tilde{\mathcal{P}}^O(x), \mathcal{V}^O(x) \rangle$.

We define the following events over this interaction:

- **Accept:** the verifier accepts, i.e., $\langle \tilde{\mathcal{P}}^O(x), \mathcal{V}^O(x) \rangle = 1$.
- **Cons:** the prover's response $\tilde{A}$ is *consistent*, i.e., $\tilde{A} = Q|_J$. Technically, we define Cons as the following two holding simultaneously: oracle answers are correct and queries agree with the honest evaluation $C_0(x)$ on J :

$$\text{Cons} := (\forall i \in J, \tilde{y}_i = O(\tilde{x}_i)) \wedge (C_0(x)|_J = \tilde{A}^{\text{in}}).$$

- **IPPAcc:** the IPP predicate accepts the *true* answers, i.e., $\phi(Q|_J) = 1$.

Our goal is to show $\Pr[\text{Accept}] \leq 2^{-\sigma}$. The analysis relies on the following three claims, which we prove below.

Claim C.2. $\Pr[\text{Accept} \wedge \neg \text{Cons}] \leq 2^{-(\sigma+2)}$.

Claim C.3. $\Pr[\text{IPPAcc}] \leq 2^{-(\sigma+2)}$.

Claim C.4. $\text{Accept} \wedge \text{Cons} \implies \text{IPPAcc}$.

Granting the claims, we first conclude the soundness analysis. Since

$$\text{Accept} \iff (\text{Accept} \wedge \neg \text{Cons}) \vee (\text{Accept} \wedge \text{Cons}),$$

Claim C.4 gives

$$\text{Accept} \implies (\text{Accept} \wedge \neg \text{Cons}) \vee \text{IPPAcc}.$$

Therefore,

$$\Pr[\text{Accept}] \leq \Pr[\text{Accept} \wedge \neg \text{Cons}] + \Pr[\text{IPPAcc}] \quad (\text{union bound}) \quad (1)$$

$$\leq 2^{-(\sigma+2)} + 2^{-(\sigma+2)} \quad (\text{Claim C.2, Claim C.3}) \quad (2)$$

$$= 2^{-(\sigma+1)} \leq 2^{-\sigma}. \quad (3)$$

It remains to prove the three claims.

Proof of Claim C.2. The verifier checks every oracle answer in $\tilde{A}$ directly: for each $i \in J$, it queries $O(\tilde{x}_i)$ and rejects unless $\tilde{y}_i = O(\tilde{x}_i)$. Therefore,

$$\Pr[\text{Accept} \wedge \neg(\forall i \in J, \tilde{y}_i = O(\tilde{x}_i))] = 0.$$

The second conjunct of Cons is exactly the DEIP statement $x \in \mathcal{L}_{J, \tilde{A}^{\text{in}}}^*$. Since the verifier accepts only when the DEIP verifier accepts, by the $2^{-(\sigma+2)}$ -soundness of the DEIP (Theorem 3.5),

$$\Pr[\text{Accept} \wedge \neg(C_0(x)|_J = \tilde{A}^{\text{in}})] \leq 2^{-(\sigma+2)}.$$

The claim follows by a union bound over the two conjuncts. $\square$

Proof of Claim C.3. We first claim that (x, Q) is ε -Hamming-far from $\mathcal{L}_{\text{acc}}$. Indeed, let Q' be any query-answer sequence for input x which differs from Q on at most an ε fraction of indices. Since $x \notin \mathcal{L}$, we have $C_1(x, Q) = C^O(x) = 0$, and by ε -robustness of C^O , $C_1(x, Q') = C_1(x, Q) = 0$. Therefore, $(x, Q') \notin \mathcal{L}_{\text{acc}}$.

Recall that the IPP for $\mathcal{L}_{\text{acc}}$ is instantiated with proximity parameter ε and soundness parameter $\sigma + 2$ (Theorem B.5), and note that $\phi(Q|_J)$ is exactly the decision of the IPP verifier when its queries to the implicit input are answered according to the true sequence Q . Therefore, by the soundness guarantee of the IPP applied to the ε -far input (x, Q) ,

$$\Pr[\text{IPPAcc}] = \Pr[\phi(Q|_J) = 1] \leq 2^{-(\sigma+2)},$$

where the probability is over the IPP interaction in step 1. $\square$

Proof of Claim C.4. Suppose that Accept and Cons both hold. By definition, $C_0(x) = Q^{\text{in}}$ is the true query sequence obtained from an honest evaluation, so the second conjunct of Cons gives $\tilde{x}_i = x_i$ for all $i \in J$, and the first conjunct then gives $\tilde{y}_i = O(\tilde{x}_i) = O(x_i) = y_i$. Hence $\tilde{A} = Q|_J$: the answers fed to the IPP predicate are exactly the answers obtained from query access to the true implicit input Q . Finally, since the verifier accepts, the verifier's check $\phi(\tilde{A}) = 1$ passes, and therefore $\phi(Q|_J) = \phi(\tilde{A}) = 1$, i.e., IPPAcc holds. $\square$

Efficiency. We obtain the claimed parameters by examining the efficiency in each step. Recall that both ingredients are instantiated with soundness parameter $\sigma + 2 = O(\sigma)$, and that $\varepsilon\ell \geq 1$ by assumption.

• **Step 1 (IPP).**

- **Circuit proven.** The IPP of Step 1 is run on the pair language $\mathcal{L}_{\text{acc}} = \{(x, Q) : C_1(x, Q) = 1\}$, decided by the oracle-free circuit C_1 followed by a single output check. Since C_1 is obtained from C by deleting the oracle gates and feeding their answers in as inputs, it has depth at most D and size at most S ; the explicit input x has length n and the implicit input Q has ℓ positions.
- **Efficiency.** Instantiating Theorem B.5 with proximity parameter ε , soundness parameter $\sigma + 2$, explicit-input length n , implicit-input length ℓ , depth D , and size S yields query complexity $q = O(\sigma/\varepsilon)$, communication $O(\sigma \cdot \varepsilon \cdot m \cdot \ell \cdot D \cdot \text{polylog}(S))$, round complexity $O(D \cdot \text{polylog}(S))$, and verifier running time $O(\sigma \cdot (\varepsilon m \ell D \text{polylog}(S) + (1/\varepsilon) \text{polylog}(S) + n \log S))$. At the end the verifier outputs the succinct description $\langle J \rangle, \langle \phi \rangle$ of the query set $J \subseteq [\ell]$ with $|J| = q = O(\sigma/\varepsilon)$ and its decision predicate, and sends $\langle J \rangle$ to the prover. Since $\langle J \rangle$ is produced in time $O(\sigma \cdot \varepsilon \cdot \ell \cdot D \cdot \text{polylog}(S))$, sending it increases the communication by at most a constant factor.

• **Step 2 (answers and DEIP).**

- **Circuit proven.** The DEIP of Step 2 is run on $\mathcal{L}_{J, A^{\text{in}}}^* = \{x \in \{0, 1\}^n : C_0(x)|_J = A^{\text{in}}\}$, decided by a circuit that evaluates C_0 (depth at most D , size at most S) and compares its J -coordinates against the hardwired string A^{in} . Here J is given by its succinct description $\langle J \rangle$, which can be produced by a logspace-uniform circuit of size $\text{poly}(|J|, \log \ell)$ and depth $\text{polylog}(\ell)$. Equality check on $|J|$ queries can be implemented in size $O(|J| \cdot m)$ and depth $\log(|J|m)$. Therefore, since $|J| \leq \ell \leq S$, the comparison can be implemented in depth $\log(m|J|) + \text{polylog}(\ell) \in \text{polylog}(S)$ and size $\text{poly}(|J|, m, \log \ell) \in \text{poly}(S)$; hence this circuit has depth $D + \text{polylog}(S)$ and size $\text{poly}(S)$, on input x of length n .
- **Efficiency.** The prover sends $A = Q|_J$, consisting of $|J| = O(\sigma/\varepsilon)$ query-answer pairs. The verifier makes $|J| = O(\sigma/\varepsilon)$ oracle queries to check $y_i = O(x_i)$ for $i \in J$ and evaluates the IPP decision predicate on A , in time $O((\sigma/\varepsilon)m \text{polylog}(\ell))$.
The parties then run the DEIP of Theorem 3.5 with soundness parameter $\sigma + 2$ on the depth- $(D + \text{polylog}(S))$, size- $\text{poly}(S)$ circuit above: this contributes communication $O(\sigma \cdot D \cdot \text{polylog}(S))$, round complexity $O(D \cdot \text{polylog}(S))$, and verifier running time $O(\sigma \cdot (n + D \text{polylog}(S)))$.

Summing the two steps, and using $\varepsilon\ell \geq 1$ to absorb the additive $O(\sigma \cdot D \cdot \text{polylog}(S))$ costs of the DEIP into the $O(\sigma \cdot \varepsilon \cdot m \cdot \ell \cdot D \cdot \text{polylog}(S))$ costs of the IPP, we obtain:

- query complexity $q = O(\sigma/\varepsilon)$;
- communication complexity $cc = O(\sigma \cdot \varepsilon m \ell D \cdot \text{polylog}(S)) + O((\sigma/\varepsilon) \cdot m \text{polylog}(\ell)) + O(\sigma \cdot D \cdot \text{polylog}(S)) = O(\sigma \cdot m \cdot \varepsilon \cdot \ell \cdot D \cdot \text{polylog}(S) + (\sigma/\varepsilon) \cdot m \cdot \text{polylog}(\ell))$;
- round complexity $O(D \cdot \text{polylog}(S))$;
- verifier running time $O(\sigma \cdot (\varepsilon m \ell D \cdot \text{polylog}(S) + (1/\varepsilon)m \text{polylog}(S) + n \log S)) + O((\sigma/\varepsilon)m \text{polylog}(\ell)) + O(\sigma \cdot (n + D \text{polylog}(S))) = O(\varepsilon\ell \cdot D + 1/\varepsilon) \cdot \sigma \cdot m \cdot \text{polylog}(S) + O(\sigma n \log S)$;
- prover running time $\text{poly}(S, \sigma)$, as the prover evaluates $C^O(x)$ in time $\text{poly}(S)$ and runs the IPP and DEIP provers, each in time $\text{poly}(S, \sigma)$.

$\square$

C.2. A Proof System for Circuits Making Adaptive Oracle Queries

Theorem C.5. Let $\varepsilon = \varepsilon(n) \in (0, 1]$, $\sigma \in \mathbb{N}$, and let $\mathcal{L}$ be a language decidable by ε -robust logspace-uniform oracle Boolean circuits C with ℓ d -adaptive oracle gates, depth D , and size S , where all gates except possibly the oracle gates have fan-in 2. Assume that query lengths are bounded by $m = \text{poly}(n)$, and that $\varepsilon \geq 1/\ell$, as otherwise ε -robustness holds vacuously (see Definition 3.3). Then there is a doubly-efficient interactive proof system for $\mathcal{L}$ with the following parameters:⁵

- soundness error $2^{-\sigma}$;
- query complexity $\tilde{O}(\sigma d/\varepsilon)$;
- communication complexity $\tilde{O}(\sigma d D m \varepsilon \ell + \sigma d m/\varepsilon)$;
- round complexity $\tilde{O}(dD)$;
- verifier running time $\tilde{O}(\sigma d(\varepsilon m \ell D + m/\varepsilon + n))$; and
- prover running time $\text{poly}(S, \sigma)$.

Proof. The protocol is given in Figure 2. Here, we analyze its completeness, soundness, and efficiency.

Completeness. If x is in the language, then the honest prover answers the queries to the inputs in the IPPs consistent with the true query-answer string Q . Completeness then follows from completeness of the IPPs and the DEIP.

Soundness. Fix any cheating prover $\tilde{\mathcal{P}}$ and $x \notin \mathcal{L}$, and let E be the event that $\mathcal{V}$ accepts. We show that $\Pr[E] \leq 2^{-\sigma}$.

We first define some helpful notations. The protocol contains d IPPs, one for each level of oracle gates in the circuit. We refer to the IPP on implicit input $(Q_1, \dots, Q_i)$ as IPP i and denote the IPP verifier for IPP i by $\mathcal{V}_i$. The IPPs in our protocol differ from standard IPPs in that the verifier does not have query access to the input: rather, the verifier asks the cheating prover $\tilde{\mathcal{P}}$ for positions in the input, and $\tilde{\mathcal{P}}$'s answers may not be consistent with the input. We use $\tilde{A}_i$ for $\tilde{\mathcal{P}}$'s string of answers A_i to queries to Q_i . Recall we use σ_{IPP} and σ_{DEIP} for the soundness parameters of the IPP and the DEIP, and set $\sigma_{\text{IPP}}, \sigma_{\text{DEIP}} = \sigma + \log 2d$.

We recursively define strings $R_1, \dots, R_d$ as follows. Set $R_1 = Q_1$. For $i = 2, \dots, d$, if there exists a tuple $(R'_1, \dots, R'_{i-1})$ such that (i) $(R'_1, \dots, R'_{i-1})$ is $\sum_{j=1}^{i-1} \varepsilon_j$ -close to $(R_1, \dots, R_{i-1})$, and (ii) each R'_j computes correctly to the checksums that the prover sent in Step 1, then define

$$R_i := (C_{i-1}(x, R'_1, \dots, R'_{i-1}), O(C_{i-1}(x, R'_1, \dots, R'_{i-1}))),$$

where $O(C_{i-1}(x, R'_1, \dots, R'_{i-1}))$ is obtained by applying O to every query in $C_{i-1}(x, R'_1, \dots, R'_{i-1})$. (If more than one such tuple exists, we define R_i using the lexicographically first one; if no such tuple exists, we set $R_i := \perp$, where by convention no query-answer string is close to, or consistent with, $\perp$.) We will show that $\tilde{\mathcal{P}}$'s answers $\tilde{A}_1, \dots, \tilde{A}_d$ are with good probability consistent with $R_1, \dots, R_d$, which means that even though our IPPs differ from standard IPPs in that the prover holds the implicit input, from $\mathcal{V}_i$'s point of view, IPP i looks the same as standard IPP on implicit input $(R_1, \dots, R_i)$.

Towards this, we define some helpful events:

- For $i \in [d]$, let F_i be the event that $\tilde{A}_i$ is consistent with R_i .
- For $i \in [d]$, let U_i be the event that there is at most one query-answer string R'_i such that R'_i is $\sum_{j=1}^d \varepsilon_j$ -Hamming-close to R_i and R'_i computes correctly to the checksums that the prover sent in step 1.

Consider the following claims:

Claim C.6. We have $\Pr[E \cap \overline{F_1}] \leq 2^{-\sigma_{\text{DEIP}}}$.

⁵We use $\tilde{O}(\cdot)$ to hide $\text{polylog}(d, S)$ factors.

DEIP for robust, adaptive setting

Input: The prover $\mathcal{P}$ and verifier $\mathcal{V}$ receive $x \in \{0, 1\}^n$, a succinct description of an ε -robust logspace-uniform oracle circuit C with ℓ d -adaptive oracle gates, depth D , and size S , and oracle access to $O : \{0, 1\}^* \rightarrow \{0, 1\}$. Note that ε is a fixed parameter, while ℓ , D , and S are parameters of the input.

Additional parameters: We let σ denote the soundness parameter. For each level $i \in [d]$, let ℓ_i be the number of oracle gates in level i , ε_i be a proximity parameter, and h_i be the checksum length. For all i , set $\varepsilon_i = \varepsilon/d$ and $h_i \leq O(\varepsilon m \ell \log \ell)$ (its exact value is determined in the analysis). We use σ_{IPP} and σ_{DEIP} for the soundness parameters of the IPP and the DEIP, and set $\sigma_{\text{IPP}}, \sigma_{\text{DEIP}} = \sigma + \log(2d)$.

Notations: For each $i \in [d]$, enumerate the oracle gates in level i by $(i, 1), \dots, (i, \ell_i)$. For $i \in [d], j \in [\ell_i]$, let $x_{i,j}$ denote the query made at oracle gate (i, j) and $y_{i,j} = O(x_{i,j})$ the corresponding answer. Let $Q_i := ((x_{i,1}, y_{i,1}), \dots, (x_{i,\ell_i}, y_{i,\ell_i}))$ denote the string of oracle query-answer pairs for oracle gates in level i . For a query-answer string S , let S^{in} denote the string of queries in S and S^{out} denote the string of answers. Decompose C^O into circuits $C_0, \dots, C_d$ without oracle gates as follows: C_0 maps $x \mapsto Q_1^{\text{in}}$; for $i \in [d-1]$, C_i maps $(x, Q_1, \dots, Q_i) \mapsto Q_{i+1}^{\text{in}}$; and C_d maps $(x, Q_1, \dots, Q_d) \mapsto C^O(x)$.

The protocol:

1. The prover claims $C^O(x) = 1$. For each $i \in [d]$, the prover computes the checksum of Q_i to h_i bits and sends the tuple of checksums $(H_i = \text{cksum}(Q_i))_{i \in [d]}$ to the verifier.
2. The prover and the verifier run the message-exchange portion of the IPP from Theorem B.5 with proximity parameter $\varepsilon_1 + \dots + \varepsilon_d$ and soundness parameter σ_{IPP} on explicit input x and implicit input $(Q_1, \dots, Q_d)$ to prove that $C_d(x, Q_1, \dots, Q_d) = 1$ and $Q_1, \dots, Q_d$ compute correctly to the checksums in step 1, where $(Q_1, \dots, Q_d)$ is the implicit part of the input. At the end of the IPP, the verifier needs to query sets of indices $J_{d,1} \subseteq [\ell_1], \dots, J_{d,d} \subseteq [\ell_d]$ in $Q_1, \dots, Q_d$, respectively.
3. The verifier asks the prover to send the query-answer pairs in Q_i at indices $J_{d,i}$ for each $i \in [d]$. For each i , the prover sends a string $A_{d,i} := Q_i|_{J_{d,i}}$, and the verifier checks that all the oracle calls in every $A_{d,i}$ are correct.
4. For $j = d-1, d-2, \dots, 1$, the prover and the verifier run an IPP with proximity parameter $\varepsilon_1 + \dots + \varepsilon_j$ and soundness parameter σ_{IPP} on input $(x, J_{j+1}, A_{j+1}^{\text{in}}, Q_1, \dots, Q_j)$ to show that $C_j(x, Q_1, \dots, Q_j)|_{J_{j+1}} = A_{j+1}^{\text{in}}$ and $Q_1, \dots, Q_j$ compute correctly to the checksums in step 1, where $J_{j+1} = \bigcup_{k=j+1}^d J_{k,j+1}$ consists of all the indices that the verifier has queried in Q_{j+1} in previous IPPs, A_{j+1}^{in} is the sorted concatenation of $A_{d,j+1}, A_{d-1,j+1}, \dots, A_{j+1,j+1}$, i.e., all the strings of answers that the prover provides to queries to Q_{j+1} in previous IPPs, and $(Q_1, \dots, Q_j)$ is the implicit part of the input. At the end, the verifier outputs sets $J_{j,1} \subseteq [\ell_1], \dots, J_{j,j} \subseteq [\ell_j]$ of indices that it wishes to query in $Q_1, \dots, Q_j$ respectively. For each $i \in [j]$, the prover sends $A_{j,i} := Q_i|_{J_{j,i}}$, and the verifier checks that the oracle calls in $A_{j,i}$ are correct.
5. Finally, letting J_1 denote $\bigcup_{k=1}^d J_{k,1}$ and A_1 denote the sorted concatenation of $A_{d,1}, \dots, A_{1,1}$, the prover and the verifier engage in a DEIP (by Theorem 3.5) with soundness parameter σ_{DEIP} to prove that $C_0(x)|_{J_1} = A_1^{\text{in}}$.
6. The verifier accepts if the IPP and DEIP verifiers accept and all of its checks pass, and rejects otherwise.

Figure 2. DEIP for robust circuits making adaptive queries

Claim C.7. For $i \in \{2, \dots, d\}$, we have

$$\Pr \left[E \cap \left(\overline{F_i} \cap \bigcap_{j=1}^{i-1} (F_j \cap U_j) \right) \right] \leq 2^{-\sigma_{\text{IPP}}}.$$

Claim C.8. We have

$$\Pr \left[E \cap \bigcap_{i=1}^d (F_i \cap U_i) \right] \leq 2^{-\sigma_{\text{IPP}}}.$$

Claim C.9. For all $i \in [d]$, with $h_i \leq O(\varepsilon m \ell \log \ell)$, we have $\Pr[E \cap \overline{U_i}] = 0$.

We first show that, assuming these claims, we have $\Pr[E] \leq 2^{-\sigma}$. We have

$$\begin{aligned} \Pr[E] &= \Pr \left[E \cap \bigcap_{i=1}^d (F_i \cap U_i) \right] + \Pr \left[E \cap \left(\bigcap_{i=1}^d \overline{(F_i \cap U_i)} \right) \right] \\ &= \Pr \left[E \cap \bigcap_{i=1}^d (F_i \cap U_i) \right] + \Pr \left[E \cap \bigcup_{i=1}^d (\overline{F_i} \cup \overline{U_i}) \right], \end{aligned}$$

where the second step applies De Morgan's. By Claim C.8, the first summand is at most $2^{-\sigma_{\text{IPP}}}$. We can bound the second summand as follows:

$$\Pr \left[E \cap \bigcup_{i=1}^d (\overline{F_i} \cup \overline{U_i}) \right] \leq \Pr[E \cap \overline{F_1}] + \sum_{i=1}^d \Pr[E \cap \overline{U_i}] + \sum_{j=2}^d \Pr \left[E \cap \left(\overline{F_j} \cap \bigcap_{k=1}^{j-1} (F_k \cap U_k) \right) \right].$$

Applying Claim C.6, Claim C.7, and Claim C.9, we get that this is at most $2^{-\sigma_{\text{DEIP}}} + (d-1) \cdot 2^{-\sigma_{\text{IPP}}} + 0$. Putting everything together, we get $\Pr[E] \leq 2^{-\sigma_{\text{DEIP}}} + d \cdot 2^{-\sigma_{\text{IPP}}} \leq 2^{-\sigma}$, as desired.

To finish, we prove the claims.

Proof of Claim C.6. Since E holds (i.e., the verifier accepts), the DEIP verifier accepts. Then since $R_1 = Q_1$, the DEIP verifies that $C_0(x)|_{J_1} = \tilde{A}_1^{\text{in}}$, and the verifier also checks the oracle calls in $\tilde{A}_1$, we have $\Pr[E \cap \overline{F_1}] \leq 2^{-\sigma_{\text{DEIP}}}$. $\square$

Proof of Claim C.7. Let $i \in \{2, \dots, d\}$. Since $F_1 \cap \dots \cap F_{i-1}$ holds, $\mathcal{V}_{i-1}$ sees answers exactly as if it had query access to the implicit input $(R_1, \dots, R_{i-1})$. Since E holds, $\mathcal{V}_{i-1}$ accepts. Thus, by IPP soundness, except with probability $2^{-\sigma_{\text{IPP}}}$, there exists a tuple $(R'_1, \dots, R'_{i-1})$ that is $\sum_{j=1}^{i-1} \varepsilon_j$ -close to $(R_1, \dots, R_{i-1})$, computes correctly to the checksums in step 1, and satisfies the relation checked by IPP $i-1$. Since $U_1 \cap \dots \cap U_{i-1}$ holds, each such R'_j is unique; in particular, $(R'_1, \dots, R'_{i-1})$ is exactly the tuple used to define R_i , which means that $R_i = (C_{i-1}(x, R'_1, \dots, R'_{i-1}), O(C_{i-1}(x, R'_1, \dots, R'_{i-1})))$. Then since $\tilde{A}_i^{\text{in}} = C_{i-1}(x, R'_1, \dots, R'_{i-1})|_{J_i}$ and the verifier checks that all the oracle calls in $\tilde{A}_i$ are correct, $\tilde{A}_i$ is consistent with R_i , so we have

$$\Pr \left[E \cap \left(\overline{F_i} \cap \bigcap_{j=1}^{i-1} (F_j \cap U_j) \right) \right] \leq 2^{-\sigma_{\text{IPP}}}.$$

$\square$

Proof of Claim C.8. Since $F_1 \cap \dots \cap F_d$ holds, the final IPP verifier $\mathcal{V}_d$ has exactly the view of a standard IPP verifier with query access to implicit input $(R_1, \dots, R_d)$. If E , then $\mathcal{V}_d$ accepts. Therefore, except with probability $2^{-\sigma_{\text{IPP}}}$, there exists a tuple $(R'_1, \dots, R'_d)$ that is $\sum_{j=1}^d \varepsilon_j$ -close to $(R_1, \dots, R_d)$, computes correctly to the checksums in step 1, and satisfies

$$C_d(x, R'_1, \dots, R'_d) = 1.$$

Since $U_1 \cap \dots \cap U_d$ holds, this tuple is unique; in particular, for each i , the prefix $(R'_1, \dots, R'_{i-1})$ is exactly the tuple used to define R_i .

Then $(R'_1, \dots, R'_d)$ is $\sum_{i=1}^d \varepsilon_i$ -close to the true query-answer string $(Q_1, \dots, Q_d)$ in the sense of Definition 3.2: starting from $R_1 = Q_1$, we can modify positions to obtain R'_1 , then recompute the queries and answers for the next level from (x, R'_1) and modify positions to obtain R'_2 , and continue in this way through level d . In total, we modify at most an $\sum_{i=1}^d \varepsilon_i$ -fraction of positions. Since C is ε -robust and $C_d(x, R'_1, \dots, R'_d) = 1$, assuming $\sum_{i=1}^d \varepsilon_i \leq \varepsilon$, we have that $C^O(x) = 1$, which is a contradiction, implying

$$\Pr \left[E \cap \bigcap_{i=1}^d (F_i \cap U_i) \right] \leq 2^{-\sigma_{\text{IPP}}}.$$

□

Proof of Claim C.9. If $\overline{U_i}$ occurs, then there exist two distinct strings R'_i, R''_i $\sum_{j=1}^d \varepsilon_j$ -Hamming-close to R_i that compute correctly to the checksums sent by the prover in step 1, i.e., $\text{cksum}(R'_i) = \text{cksum}(R''_i)$.

Choosing $h_i \leq O(\varepsilon \ell \cdot m \log \ell)$ (e.g., we can set $h_i = 4\varepsilon \ell \cdot m \log \ell$), which guarantees ε -unique decoding checksums by Lemma B.8, we get that $\Pr[\overline{U_i}] = 0$, which implies $\Pr[E \cap \overline{U_i}] = 0$.

□

Efficiency. Finally, we analyze the efficiency of the protocol. For $i \in [d]$, we take $\varepsilon_i = \varepsilon/d$ and $h_i = O(\varepsilon m \ell \log \ell)$.

We break the protocol into the following parts:

- At the start of the protocol, the prover sends a checksum of each level of oracle gates to the verifier. This involves $O(1)$ rounds and communication

$$\sum_{i=1}^d h_i \leq O(d \cdot \varepsilon \ell \log \ell \cdot m).$$

- The protocol contains d IPPs. We refer to the first IPP as IPP- d and the second IPP as IPP- $(d-1)$ and so on. For $i \in [d-1]$, IPP- i is run on language $\mathcal{L}_{J_{i+1}, A_{i+1}^{\text{in}}} := \{x \in \{0, 1\}^n \mid C_i(x, Q_1, \dots, Q_i)|_{J_{i+1}} = A_{i+1}^{\text{in}} \text{ and } \text{cksum}(Q_j) = H_j \text{ for all } j \in [i]\}$, which is decided by a circuit that runs C_i on input $(x, Q_1, \dots, Q_i)$, does an equality check to verify that its output at the indices in J_{i+1} matches A_{i+1}^{in} , and evaluates the checksums of $Q_1, \dots, Q_i$ and compares them to the checksums $H_1, \dots, H_i$ sent in step 1. Since each checksum is an $\mathbb{F}$ -linear map of its input, it can be evaluated with $\text{poly}(S)$ gates in $\text{polylog}(S)$ depth, so such a circuit can be implemented with $\text{poly}(S)$ gates and $O(D + \text{polylog}(S))$ depth. IPP- d is run on the language $\{x \in \{0, 1\}^n \mid C_d(x, Q_1, \dots, Q_d) = 1 \text{ and } \text{cksum}(Q_j) = H_j \text{ for all } j \in [d]\}$, and this can be decided by a circuit that runs C_d on input $(x, Q_1, \dots, Q_d)$, checks that the output is 1, and checks the checksums in the same way. Such a circuit can also be implemented with $O(\text{poly}(S))$ gates and $O(D + \text{polylog}(S))$ depth. Additionally, for $i \in [d]$, IPP- i has explicit input $(x, J_{i+1}, A_{i+1}^{\text{in}}, H_1, \dots, H_i)$ (just $(x, H_1, \dots, H_d)$ for $i = d$), whose length beyond n only contributes terms dominated by the other costs accounted below, implicit input length $\sum_{j=1}^i \ell_j$, proximity parameter $\sum_{j=1}^i \varepsilon_j = i\varepsilon/d$, and soundness parameter $\sigma_{\text{IPP}} = \sigma + \log 2d$. Let

$$L_i := \sum_{j=1}^i \ell_j, \quad \alpha_i := \sum_{j=1}^i \varepsilon_j = \frac{i\varepsilon}{d}, \quad \sigma_{\text{IPP}} := \sigma + \log(2d).$$

Applying Theorem B.5 and summing over all d IPPs, we get that the IPPs contribute the following towards the efficiencies:

- query complexity

$$\sum_{i=1}^d O\left(\frac{\sigma_{\text{IPP}}}{\alpha_i}\right) = O\left(\frac{\sigma_{\text{IPP}} d \log d}{\varepsilon}\right).$$

- communication complexity

$$\begin{aligned} & \sum_{i=1}^d O(\sigma_{\text{IPP}} \alpha_i m L_i (D + \text{polylog}(S)) \text{polylog}(S) + \sigma_{\text{IPP}} / \alpha_i \cdot m) \\ &= O(\sigma_{\text{IPP}} \varepsilon m d (D + \text{polylog}(S)) \text{polylog}(S) + \sigma_{\text{IPP}} d \log d \cdot m / \varepsilon), \end{aligned}$$

where we use that $L_i \leq \ell$ for every i and $\sum_{i=1}^d 1/\alpha_i = O((d \log d)/\varepsilon)$,

– round complexity

$$O(d(D + \text{polylog}(S))\text{polylog}(S)),$$

– verifier running time

$$\sum_{i=1}^d O\left(\sigma_{\text{IPP}} \left(\alpha_i m L_i (D + \text{polylog}(S)) \text{polylog}(S) + \frac{m}{\alpha_i} \text{polylog}(S) + n \log S \right)\right),$$

which is

$$O\left(\sigma_{\text{IPP}} \left(\frac{\varepsilon(D + \text{polylog}(S))\text{polylog}(S)}{d} \sum_{i=1}^d i L_i \cdot m + \frac{d \log d}{\varepsilon} m \text{polylog}(S) + dn \log S \right)\right),$$

which, simplifying, is

$$O\left(\sigma_{\text{IPP}} \left(\varepsilon m \ell d (D + \text{polylog}(S)) \text{polylog}(S) + \frac{d \log d}{\varepsilon} m \text{polylog}(S) + dn \log S \right)\right),$$

and

– prover running time $\text{poly}(S, \sigma_{\text{IPP}}, d)$, which is $\text{poly}(S, \sigma)$.

At the end, the verifier needs to send the prover the queries that he wants to make to the implicit input. This cost is swallowed by the communication complexity of the IPP.

- The protocol contains one DEIP, run on the language $\mathcal{L}_{J_1, A_1^{\text{in}}} := \{x \in \{0, 1\}^n \mid C_0(x)|_{J_1} = A_1^{\text{in}}\}$, which can be decided by a circuit that runs C_0 on x and performs an equality check to verify that its output at the indices in J_1 matches A_1^{in} . Such a circuit can be implemented with $O(\text{poly}(S))$ gates and $O(D + \text{polylog}(S))$ depth. Applying Theorem 3.5 with soundness parameter $\sigma_{\text{DEIP}} = \sigma + \log 2d$, we get that the DEIP contributes the following:

– communication complexity

$$O(\sigma_{\text{DEIP}}(D + \text{polylog}(S))\text{polylog}(S))$$

– round complexity

$$O((D + \text{polylog}(S))\text{polylog}(S))$$

– verifier running time

$$O(\sigma_{\text{DEIP}}(n + (D + \text{polylog}(S))\text{polylog}(S)))$$

– prover running time

$$\text{poly}(S, \sigma)$$

Putting everything together, using $\tilde{O}(\cdot)$ to hide $\text{polylog}(d)$ and $\text{polylog}(S)$ factors, and using $\varepsilon \ell \geq 1$ to absorb the additive costs of the DEIP into the costs of the IPPs, we get the following complexities:

- query complexity

$$O\left(\frac{(\sigma + \log d)d \log d}{\varepsilon}\right) = \tilde{O}\left(\frac{\sigma d}{\varepsilon}\right)$$

- communication complexity

$$\begin{aligned} &O(d(\varepsilon m \ell \log \ell) + (\sigma + \log d)\varepsilon m \ell d(D + \text{polylog}(S))\text{polylog}(S) + (\sigma + \log d)d \log d \cdot m/\varepsilon) \\ &= \tilde{O}(\sigma d D m \varepsilon \ell + \sigma d m/\varepsilon) \end{aligned}$$

- round complexity

$$O(d(D + \text{polylog}(S))\text{polylog}(S)) = \tilde{O}(dD)$$

- verifier running time

$$O\left((\sigma + \log d) \left(\varepsilon m \ell d (D + \text{polylog}(S)) \text{polylog}(S) + \frac{d \log d}{\varepsilon} m \text{polylog}(S) + dn \log S\right)\right) \\ = \tilde{O}\left(\sigma d \left(\varepsilon m \ell D + \frac{m}{\varepsilon} + n\right)\right)$$

- prover running time

$$\text{poly}(S, \sigma)$$

□

C.3. An Argument System for Circuits Making Adaptive Oracle Queries

Theorem C.10. Let $\varepsilon = \varepsilon(n) \in (0, 1]$ be a proximity parameter, and let $\mathcal{L}$ be a language decidable by ε -robust logspace-uniform oracle Boolean circuits with $\ell = \ell(n)$ oracle gates and of polynomial size $S = S(n) \leq 2^\lambda$. Assume all query lengths are bounded by $m = \text{poly}(n)$. Then there is a doubly-efficient interactive argument system for $\mathcal{L}$ with the following parameters:

- soundness error $1/2$ for any PPT adversary;
- query complexity $q = O(1/\varepsilon)$;
- communication complexity $cc = (\text{poly}(\lambda) + m) \cdot 1/\varepsilon$;
- round complexity $O(1)$;
- verifier running time $\text{poly}(\lambda) \cdot (1/\varepsilon + n) + m/\varepsilon$; and
- prover running time $\text{poly}(S, \lambda) \cdot 1/\varepsilon$.

Proof. The protocol appears in Figure 3. Here we analyze its efficiency, completeness, and soundness.

Efficiency. The communication is composed of:

- Public parameters pp_{Kil} , hk of size $\text{poly}(\lambda)$.
- Hash root rt of size $\text{poly}(\lambda)$.
- The Kilian's protocol interaction of size $\text{poly}(\lambda)$.
- q oracle input-output pairs and their openings $\{(x_j^*, y_j^*, \rho_j)\}_{j \in J}$, where $q = O(1/\varepsilon)$. Their total size is $(\text{poly}(\lambda) + m) \cdot 1/\varepsilon$.

Thus the total communication is $(\text{poly}(\lambda) + m) \cdot 1/\varepsilon$.

During the above protocol, the verifier runs in time $\text{poly}(\lambda) \cdot (1/\varepsilon + n) + m/\varepsilon$ and the prover runs in time $\text{poly}(S, \lambda) \cdot 1/\varepsilon$.

Completeness. The completeness follows from the completeness of Kilian's protocol and the hash tree.

Soundness. Let $\tilde{\mathcal{P}}$ be any poly-time malicious prover, and we rewind $\tilde{\mathcal{P}}$ (just after sending rt) to obtain a prover for the succinct argument of knowledge $\tilde{\mathcal{P}}_1$. Specifically, we run $\text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1}$ to extract $\{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}$ that is consistent with rt and the computation. By knowledge soundness, we have

$$\Pr \left[\begin{array}{c} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle = 1 \wedge \\ (\text{rt} \neq \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \\ \vee C^{x_i^* \rightarrow y_i^*}(x) = 0) \end{array} : \begin{array}{c} \text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda), \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, \ell) \\ \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\} \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda).$$

Now we claim that for at most an ε fraction of $\{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}$, it holds that $y_i^* \neq O(x_i^*)$.

Doubly-efficient interactive argument for robust, adaptive setting

Input: The prover $\mathcal{P}$ and verifier $\mathcal{V}$ receive $x \in \{0, 1\}^n$, a succinct description of a logspace-uniform oracle circuit C containing ℓ oracle gates, and oracle access to $O : \{0, 1\}^* \rightarrow \{0, 1\}$.

Additional parameters: We take $q = 2/\varepsilon$.

Notations and tools: Enumerate the oracle gates of C^O in topological order by $1, \dots, \ell$. On input x , for each $i \in [\ell]$, let x_i denote the input to the i th oracle gate and $y_i = O(x_i)$ the corresponding oracle answer. Let $\mathcal{M}$ denote the set of oracle input-output pairs. We will use the following tools:

- A succinct argument of knowledge $\text{KIL} = (\text{Setup}_{\text{Kil}}, \mathcal{P}_{\text{Kil}}, \mathcal{V}_{\text{Kil}}, \text{Ext}_{\text{Kil}})$ for NP relations (see Section B.4).
- A hash tree family $\text{HT} = (\mathcal{G}_{\text{ht}}, \mathcal{H}_{\text{ht}}, \mathcal{P}_{\text{ht}}, \mathcal{V}_{\text{ht}})$ (see Definition B.14).

The protocol:

1. The verifier generates public parameters $\text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda)$, and $\text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, \ell)$, where hk are hash keys for database over $\mathcal{M}$.
2. The prover simulates the entire execution $C^O(x) = 1$, computing all intermediate oracle queries $x_1, \dots, x_\ell$ and answers $y_1 = O(x_1), \dots, y_\ell = O(x_\ell)$. The prover then computes the hash tree root $\text{rt} \leftarrow \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1, y_1), \dots, (x_\ell, y_\ell)\})$ and sends rt to the verifier.
3. The prover and verifier engage in Kilian's protocol to prove that there exists a set $\{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}$ that is (i) consistent with rt and (ii) on input x , taking the oracle answers to be those in $\{y_i^*\}_{i \in [\ell]}$, C makes the queries $\{x_i^*\}_{i \in [\ell]}$ and accepts. In particular, they prove the following relation

$$\mathcal{R} := \{(\text{hk}, (C, x, \text{rt}), \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \mid \text{rt} = \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \wedge C^{x_i^* \rightarrow y_i^*}(x) = 1\},$$

where $C^{x_i^* \rightarrow y_i^*}(x) = 1$ means all queries made by $C(x)$ are $\{x_i^*\}_{i \in [\ell]}$ and given that all query answers are $\{y_i^*\}_{i \in [\ell]}$, C accepts.

4. The verifier randomly samples q indices $J \subseteq [\ell]$ and asks the prover to open the hash tree on these indices. The prover sends $\{(x_j^*, y_j^*, \rho_j)\}_{j \in J}$, where ρ_j is the hash tree opening proof for (x_j^*, y_j^*) . The verifier checks that for all $j \in J$, $\mathcal{V}_{\text{ht}}(\text{hk}, \text{rt}, j, (x_j^*, y_j^*), \rho_j) = 1$ and $y_j^* = O(x_j^*)$.
5. The verifier accepts if all checks pass, and rejects otherwise.

Figure 3. Doubly-efficient interactive argument for robust circuits making adaptive queries

Claim C.11. For any poly-time malicious prover $\tilde{\mathcal{P}}$, it holds that

$$\Pr \left[\begin{array}{l} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle = 1 \\ \wedge \text{rt} = \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \\ \wedge |\{i \in [\ell] : y_i^* \neq O(x_i^*)\}| > \varepsilon \cdot \ell \end{array} : \begin{array}{l} \text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda), \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, \ell) \\ \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\} \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \frac{1}{3}$$

Proof. We consider two events:

- E_0 : the sampled q indices J contains at least one index j such that $y_j^* \neq O(x_j^*)$.
- E_1 : the sampled q indices J contains no index j such that $y_j^* \neq O(x_j^*)$.

If event E_0 happens and the verifier accepts, then $\tilde{\mathcal{P}}$ must break the binding property of the hash tree. That is,

$$\Pr \left[\begin{array}{l} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle = 1 \\ \wedge \text{rt} = \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \\ \wedge |\{i \in [\ell] : y_i^* \neq O(x_i^*)\}| > \varepsilon \cdot \ell \\ \wedge E_0 \end{array} : \begin{array}{l} \text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda), \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, \ell) \\ \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\} \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda).$$

On the other hand, the probability for E_1 to happen is small. In particular, $\Pr[E_1] \leq (1 - \varepsilon)^q$. Since we set $q = \frac{2}{\varepsilon}$, we have $\Pr[E_1] \leq 1/e^2$.

Combining the two cases we have

$$\Pr \left[\begin{array}{l} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle = 1 \\ \wedge \text{rt} = \mathcal{H}_{\text{ht}}(\text{hk}, \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\}) \\ \wedge |\{i \in [\ell] : y_i^* \neq O(x_i^*)\}| > \varepsilon \cdot \ell \end{array} : \begin{array}{l} \text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda), \text{hk} \leftarrow \mathcal{G}_{\text{ht}}(1^\lambda, \ell) \\ \{(x_1^*, y_1^*), \dots, (x_\ell^*, y_\ell^*)\} \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda) + \frac{1}{e^2} \leq \frac{1}{3}.$$

□

Therefore, if $\tilde{\mathcal{P}}$ convinces the verifier, then it must be the case that $C^O(x)$ accepts with at most an ε fraction of incorrect oracle answers, that is, with probability at most $1/3 + \text{negl}(\lambda) \leq 1/2$, we have $C^{x_i^* \rightarrow y_i^*}(x) = 1$ and $|\{i \in [\ell] : y_i^* \neq O(x_i^*)\}| \leq \varepsilon \cdot \ell$, which contradicts with the fact that C is ε -robust and $x \notin \mathcal{L}$. This finishes our soundness analysis. □

D. A Doubly-Efficient Interactive Argument for Low-Degree Oracles

Here we present our doubly-efficient argument system for circuits with access to a low-degree oracle.

Definition D.1 (Degree of Oracle). Let $O : \{0, 1\}^* \rightarrow \{0, 1\}$ be an oracle and O_n be the restriction of O to inputs of length at most n . We say that O has degree $d(n)$ if for every $n \in \mathbb{N}$, there exists a degree- $d(n)$ polynomial P_n over some finite field $\mathbb{F}$ of size $2^{O(n)}$, such that we can encode every $x \in \{0, 1\}^{\leq n}$ into $\mathbb{F}$ and $O_n(x) = P_n(x)$ for every such x .

Remark D.2 (Oracle Access at Field Points). Throughout this section, we adopt the convention that oracle access to a degree- $d(n)$ oracle O means oracle access to P_n : the parties may query the oracle at any point $z \in \mathbb{F}$ and receive $P_n(z)$, which on encoded binary inputs coincides with O .

Remark D.3 (Field Size). Note that $\log |\mathbb{F}| \geq m$ always holds: the encoding of $\{0, 1\}^{\leq m}$ into $\mathbb{F}$ is injective, and if $\log |\mathbb{F}| < m$, then field elements would be shorter descriptions of the queries than the queries themselves, which is impossible by counting.

Theorem D.4. Let $O : \{0, 1\}^* \rightarrow \{0, 1\}$ be an oracle with degree $d(n) = \text{poly}(n)$, and let $\mathcal{L}$ be a language decidable by logspace-uniform oracle Boolean circuits with $S = \text{poly}(n)$ gates. Take λ as the security parameter and m as the maximum query length to the oracle. We assume that the field $\mathbb{F}$ over which O_m is represented satisfies $\log |\mathbb{F}| \geq \lambda$. Assuming the polynomial hardness of SIS, there exists an interactive argument system for $\mathcal{L}$ with the following parameters:

- query complexity $q = 1$;
- communication complexity $cc = \text{poly}(\lambda) \cdot m$;

- round complexity $O(\log n)$;
- verifier running time $\text{poly}(\lambda) \cdot (n + m)$; and
- prover running time $\text{poly}(\lambda, S)$.

Proof. The protocol is stated in Fig. 4. We analyze its efficiency, completeness, and soundness as follows.

Efficiency. The communication of this protocol is composed of:

- Public parameters pp of size $\text{poly}(\lambda)$.
- Commitments (c_G, c_F) of size $\text{poly}(\lambda)$.
- Kilian's protocol communication, of size $\text{poly}(\lambda)$.
- The challenge u of size $\log |\mathbb{F}| = O(m + \lambda)$.
- The openings (v_G, v_F) of size $O(m + \lambda)$.
- The polynomial commitment opening, of size $\text{poly}(\lambda) \cdot m$.

Therefore, the total communication is $m \cdot \text{poly}(\lambda)$.

For round complexity, since the Kilian's protocol has $O(1)$ rounds, and PC local opening argument has $O(\log(S \cdot d(m))) = O(\log n)$ rounds, the total number of rounds is $O(\log n)$.

The computation of the verifier is composed of Kilian's protocol verifier, polynomial commitment verifier and checking $v_F = O(v_G)$. Therefore the running time is $\text{poly}(\lambda) \cdot (n + m)$, and the query complexity is 1.

Completeness. Completeness follows from completeness of the polynomial commitment scheme and Kilian's protocol.

Soundness. Let $\tilde{\mathcal{P}}$ be any malicious prover. To prove computational soundness of this protocol, we rewind $\tilde{\mathcal{P}}$ (just after sending (c_G, c_F)) to obtain a prover for the succinct argument of knowledge $\tilde{\mathcal{P}}_1$, and extract the witness from the invocation of Kilian's protocol. Specifically, we run $\text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1}$ to extract $(G^*, \text{st}_G^*, F^*, \text{st}_F^*)$.

Since the protocol also performs Kilian's protocol verification, by definition of knowledge extraction, we know that if $\tilde{\mathcal{P}}$ convinces the verifier, then G^*, F^* extracted by $\text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1}$ must be valid, in the sense that they are the polynomials underlying c_G, c_F , and $C(x)$ accepts when making queries by G^* and using oracle answers from F^* , which we denote by $C^{G^* \rightarrow F^*}(x) = 1$.

Claim D.5. For any poly-time prover $\tilde{\mathcal{P}}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[\begin{array}{l} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle(\text{pp}, C, x) = 1 \wedge \\ (\text{Open}(\text{pp}_{\text{CM}}^G, c_G, G^*, \text{st}_G^*) = 0 \vee \text{Open}(\text{pp}_{\text{CM}}^F, c_F, F^*, \text{st}_F^*) = 0) \\ \vee C^{G^* \rightarrow F^*}(x) = 0 \end{array} : \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, S) \\ (G^*, \text{st}_G^*, F^*, \text{st}_F^*) \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda).$$

Then we claim that our protocol ensures $F^* = O(G^*)$ with all but negligible probability.

Claim D.6. For any poly-time prover $\tilde{\mathcal{P}}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[\langle \tilde{\mathcal{P}}, \mathcal{V} \rangle(\text{pp}, C, x) = 1 \wedge F^* \neq O(G^*) : \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, S) \\ (G^*, \text{st}_G^*, F^*, \text{st}_F^*) \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda).$$

Doubly-efficient interactive argument for circuits with access to a low-degree oracle

Input: The prover $\mathcal{P}$ and verifier $\mathcal{V}$ receive $x \in \{0, 1\}^n$, a succinct description of an oracle circuit C of size S , and oracle access to $O : \{0, 1\}^* \rightarrow \{0, 1\}$ with degree $d(n) = \text{poly}(n)$.

Notations: Let $m = m(\lambda)$ be the maximum input length to the oracle. Let $\mathbb{F} = \mathbb{F}_\lambda$ be the field over which O_m is represented as a polynomial, and assume that $\log |\mathbb{F}| \geq \max\{m, \lambda\}$.

Ingredients: We make use of the following tools:

- A polynomial commitment scheme $\text{PC} = (\text{Setup}_{\text{CM}}, \text{Commit}, \text{Open}, \text{Setup}_{\text{IP}}, \mathcal{P}_{\text{PC}}, \mathcal{V}_{\text{PC}})$ with efficient opening and verification for degree-bounded polynomials over $\mathbb{F}$, as in Theorem B.20, instantiated with degree bound S for committing to G and degree bound $S \cdot d(m)$ for committing to F .
- A succinct argument of knowledge $\text{KIL} = (\text{Setup}_{\text{Kil}}, \mathcal{P}_{\text{Kil}}, \mathcal{V}_{\text{Kil}}, \text{Ext}_{\text{Kil}})$ for NP relations, as in Section B.4.

The protocol:

1. The setup algorithm samples $\text{pp}_{\text{CM}}^G \leftarrow \text{Setup}_{\text{CM}}(1^\lambda, S)$, $\text{pp}_{\text{CM}}^F \leftarrow \text{Setup}_{\text{CM}}(1^\lambda, S \cdot d(m))$, $\text{pp}_{\text{IP}} \leftarrow \text{Setup}_{\text{IP}}(1^\lambda)$, and $\text{pp}_{\text{Kil}} \leftarrow \text{Setup}_{\text{Kil}}(1^\lambda)$, and outputs $\text{pp} = (\text{pp}_{\text{CM}}^G, \text{pp}_{\text{CM}}^F, \text{pp}_{\text{IP}}, \text{pp}_{\text{Kil}})$.
2. The prover simulates the accepting computation $C^O(x) = 1$ and records the oracle queries $x_1, \dots, x_S$ and corresponding answers $y_i = O(x_i)$ for every $i \in [S]$. It constructs a degree- S polynomial G satisfying $G(i) = x_i$ for every $i \in [S]$, and the unique degree- $S \cdot d(m)$ polynomial F satisfying $F(z) = O(G(z))$ for every $z \in \mathbb{F}$ (here we naturally encode oracle outputs in $\mathbb{F}$); constructing F requires at most $S \cdot d(m)$ oracle queries. The prover computes

$$(c_G, \text{st}_G) \leftarrow \text{Commit}(\text{pp}_{\text{CM}}^G, G) \quad \text{and} \quad (c_F, \text{st}_F) \leftarrow \text{Commit}(\text{pp}_{\text{CM}}^F, F),$$

and sends (c_G, c_F) to the verifier.

3. The prover and verifier engage in Kilian's protocol for the relation

$$\mathcal{R} = \{((\text{pp}_{\text{CM}}^G, \text{pp}_{\text{CM}}^F, C, x, c_G, c_F), (G, \text{st}_G, F, \text{st}_F)) : \text{Open}(\text{pp}_{\text{CM}}^G, c_G, G, \text{st}_G) = 1, \\ \text{Open}(\text{pp}_{\text{CM}}^F, c_F, F, \text{st}_F) = 1 \wedge C^{G \rightarrow F}(x) = 1\},$$

where $C^{G \rightarrow F}(x) = 1$ means that, on input x , the i th oracle query made by C is $G(i)$, its answer is taken to be $F(i)$, and the resulting computation accepts. The verifier rejects if the Kilian verifier rejects.

4. The verifier samples $u \leftarrow \mathbb{F}$ uniformly at random and sends u to the prover. The prover responds with $v_G = G(u)$ and $v_F = F(u)$.
5. The prover and verifier run the polynomial commitment opening protocol with public parameters pp_{IP} to prove that the polynomials committed by c_G and c_F evaluate to v_G and v_F , respectively, at u . The verifier rejects if the polynomial commitment verifier rejects.
6. The verifier accepts if the Kilian verifier and polynomial commitment verifier accept and $v_F = O(v_G)$, and rejects otherwise.

Figure 4. Doubly-efficient interactive argument for circuits with access to a low-degree oracle

Proof. We bound the probability by considering whether $F^*(u) = O(G^*(u))$.

First, a valid opening under pp_{CM}^G (resp. pp_{CM}^F) enforces $\deg(G^*) \leq S$ (resp. $\deg(F^*) \leq S \cdot d(m)$), so both F^* and $O \circ G^*$ are polynomials over $\mathbb{F}$ of degree at most $S \cdot d(m)$. Hence, if $F^* \neq O(G^*)$, by Schwartz-Zippel they can agree on at most $S \cdot d(m)$ points. Therefore, as u is sampled uniformly from $\mathbb{F}$, we have

$$\Pr \left[\begin{array}{c} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle(\text{pp}, C, x) = 1 \\ \wedge F^* \neq O(G^*) \wedge F^*(u) = O(G^*(u)) \end{array} : \begin{array}{c} \text{pp} \leftarrow \text{Setup}(1^\lambda, S) \\ (G^*, \text{st}_G^*, F^*, \text{st}_F^*) \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \frac{S \cdot d(m)}{2^\lambda}.$$

Otherwise, if $F^*(u) \neq O(G^*(u))$, then $\tilde{\mathcal{P}}$ must break the polynomial commitment binding property to convince the verifier. In particular, either $v_G \neq G^*(u)$ or $v_F \neq F^*(u)$. Therefore, we can construct an adversary that breaks the binding property, by honestly computing $G^*(u)$, $F^*(u)$ and providing their openings. This gives us that for some $\text{negl}(\cdot)$,

$$\Pr \left[\begin{array}{c} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle(\text{pp}, C, x) = 1 \\ \wedge F^* \neq O(G^*) \wedge F^*(u) \neq O(G^*(u)) \end{array} : \begin{array}{c} \text{pp} \leftarrow \text{Setup}(1^\lambda, S) \\ (G^*, \text{st}_G^*, F^*, \text{st}_F^*) \leftarrow \text{Ext}_{\text{Kil}}^{\tilde{\mathcal{P}}_1} \end{array} \right] \leq \text{negl}(\lambda).$$

Combining the two cases, we conclude the proof. □

Now since $C^{G^* \rightarrow F^*}(x) = 1$ and $F^* = O(G^*)$, we have that $C^O(x) = 1$. Therefore,

$$\Pr \left[\begin{array}{c} \langle \tilde{\mathcal{P}}, \mathcal{V} \rangle(\text{pp}, C, x) = 1 \wedge \\ C^O(x) = 0 \end{array} : \text{pp} \leftarrow \text{Setup}(1^\lambda, S) \right] \leq \text{negl}(\lambda).$$

□